Scaling GPU KV-Cache Efficiency and Inference Concurrency for Autonomous Agents with HashedContext: A Dual-Mode Architecture

Rahat Hasan
Akkhar-Labs
Bogura, Bangladesh
rahathasan5773@gmail.com

Abstract

Autonomous agentic coding systems routinely transmit source files to Large Language Models (LLMs) across multi-turn software development workflows. In conventional architectures, unchanged files are re-read from storage and redundantly re-serialized into append-only conversation histories on every interaction step. For M files of N tokens over P turns, each prompt grows as $\mathcal{O}(P \cdot M \cdot N)$ and cumulative prompt tokens as $\mathcal{O}(P^2 \cdot M \cdot N)$, which saturates GPU High-Bandwidth Memory (HBM) key-value (KV) cache allocations and degrades multi-tenant serving concurrency.

We introduce **HashedContext**, a content-addressed context architecture that removes redundant file copies from multi-turn agent prompts. It is built from SHA-256 hash-gated file tracking, compact `[HASH_STUB]` references ($\approx$67 tokens), and a local content-addressed vault with on-demand resolution $\mathcal{R}(h)$. HashedContext is a **Dual-Mode Architecture**. *Mode A (Authoritative Tail)* replaces every historical file copy with a stub and places one complete, self-attesting copy of each current file at the end of the prompt. *Mode B (Lazy Tail)* never rewrites history: each file is sent in full once, unchanged re-reads become stubs, and changes are sent as line diffs, with the full file resent after at most 10 diffs. Both modes are lossless and can run in client editors or at inference gateways.

Simulated validation. We evaluate both modes with a reference implementation driven by 20,000 scripted coding agents on four real open-source repositories (824,728 turns, exact BPE token counts) and with a GPU serving simulator using H100 constants (80 GB HBM, 3.35 TB/s, 70B-class FP8 model). Every session reconstructs byte-exact. Mode A reduces the KV-cache an agent needs by 88.7% compared with append-only context (69.0% compared with append-only plus compaction); Mode B by 84.8% (58.2%). The modes differ in compute. With a warm per-session prefix cache, Mode B processes 88.1% fewer prompt tokens than append-only, while Mode A processes 3.6% more because it moves the live file copy each turn; with no cache, Mode A processes 88.7% fewer and Mode B 84.8% fewer. On a simulated 16-GPU H100 cluster serving 512 agents, where memory pressure evicts the cache, Mode A completes 3.57× and Mode B 3.21× more agent turns per hour than append-only with compaction, and they cut p95 latency 72% and 70%. On a single GPU with prefix caching, Mode B completes 1.07–1.76× the turns of Mode A at 1–8 agents per GPU, while Mode A leads at 16 or more agents or without a cache. We therefore recommend Mode B when the prefix cache stays warm, and Mode A under heavy load, without caching, or where the model should always see one complete current copy. Cumulative cost remains quadratic in the number of turns, with a coefficient roughly 25–30× smaller. GPU results come from simulation, not physical hardware. In a live test on `deepseek-v4.1-flash` (30 cases, a file with 10 edits), Mode B answered 90.0% of questions about the file's current content correctly with a 10-diff chain and 93.3% with `max_diffs_before_rebase` = 5, against 93.3% for Mode A and 96.7% for append-only. These differences are one or two cases and within sampling noise; we recommend $N = 5$ for accuracy-critical work pending larger, multi-model tests.

1 Introduction

The emergence of autonomous agentic coding frameworks (e.g., Devin, Claude Code, Cursor, Cline) has shifted large language model interaction from single-turn completion to long-horizon, multi-turn state mutation. In a representative debugging session, an autonomous agent iteratively inspects source trees, formulates hypotheses, executes edits, evaluates test suites, and verifies syntax over dozens of sequential conversational turns.

Despite significant operational capabilities, modern agentic systems suffer from a critical architectural inefficiency: **unbounded redundant context serialization**. Because autonomous agents lack cryptographic confidence in file system immutability across turns, existing tools routinely re-read and append full file contents into conversation histories prior to every action. For a repository working set of M files averaging N tokens over P turns, the prompt at turn P carries $\mathcal{O}(M \times N \times P)$ file tokens, and the cumulative prompt load over the session scales as $\mathcal{O}(M \times N \times P^2)$.

This excessive re-transmission causes profound systemic

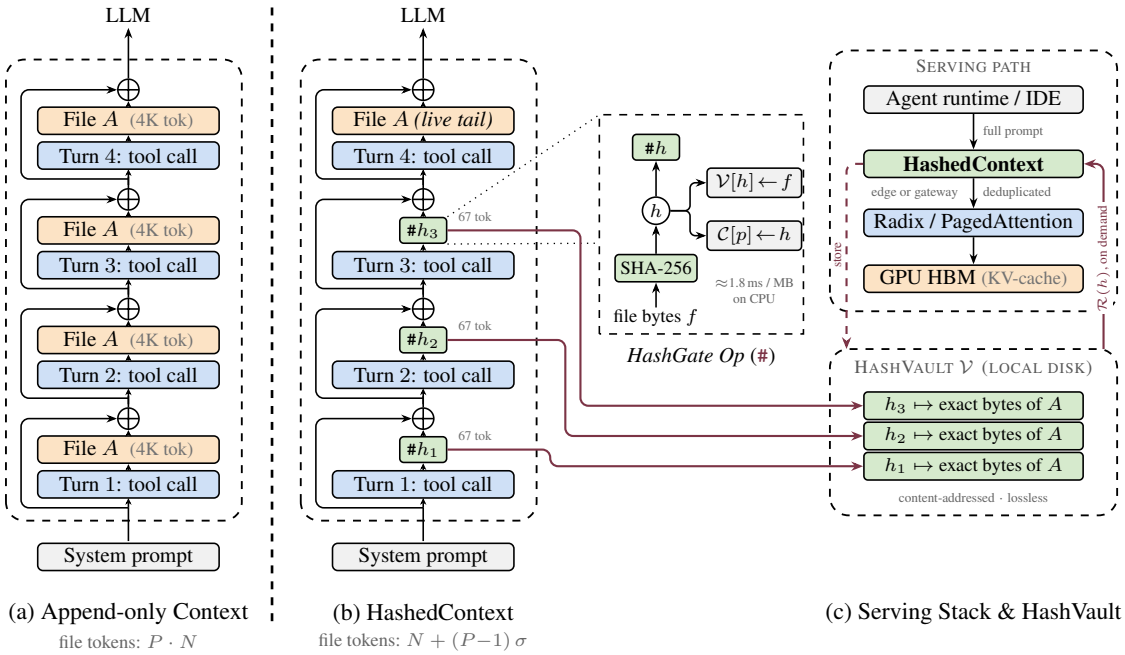


Figure 1: Overview of HashedContext. (a) Conventional agents append every file read to an append-only log, so an unchanged file is resident in the KV-cache once per turn. (b) HashedContext replaces each historical copy with a ∼67-token SHA-256 stub (#) and materializes one authoritative copy at the prompt tail; reasoning and diffs stay verbatim. (c) Historical bytes live in a local content-addressed vault and are resolved for a single turn via $\mathcal{R}(h)$, so GPU memory tracks the working set, not the session length.

strain:

1. **KV-Cache Saturation and VRAM Waste:** Modern model serving engines allocate gigabytes of scarce high-bandwidth memory (HBM3) to retain Key-Value (KV) cache vectors. At scale, long contexts saturate GPU memory capacity, forcing frequent context eviction or high-latency offloading to host RAM and NVMe storage.

2. **Attention Dilution and Version Regression:** Transformer self-attention mechanisms exhibit performance degradation (the "lost-in-the-middle" phenomenon) when multiple conflicting, partially edited copies of the same source file populate the prompt history. Models frequently attend to historical bugs already resolved in earlier turns.

3. **Memory-Bandwidth Inference Throttling:** Autoregressive decoding speed is fundamentally memory-bandwidth bound. Streaming hundreds of thousands of historical Key/Value vectors from HBM into SRAM on every generated token substantially increases Time-Per-Output-Token (TPOT).

While server-side techniques like PagedAttention [1] have revolutionized *spatial memory allocation* by partitioning physical VRAM into non-contiguous blocks, they do not reduce the *temporal redundancy* of the input sequence itself. A 100,000-token prompt still consumes 100,000 tokens of KV-cache, regardless of how efficiently those blocks are mapped.

To address this challenge, we present **HashedContext**, a content-addressed prompt architecture designed to decouple session horizon length from token memory footprint. Inspired by Content-Addressed Storage (CAS) principles employed in cryptographic state tracking systems, HashedContext replaces redundant historical file observations with lightweight cryptographic hash stubs (`[HASH_STUB]`), while rebasing the single authoritative, live representation of the files at the prompt's tail (Figure 1). Because the best prompt layout depends on whether the serving stack keeps a warm prefix cache, HashedContext offers two modes with different trade-offs: Mode A keeps one complete copy of each current file at the tail, and Mode B keeps history append-only and sends changes as line diffs (Section 3.7).

Contributions. Our primary contributions are:

- **The HashedContext Architecture:** A formal protocol combining SHA-256 content-addressing, historical stubbing, and attention-anchored tail rebasing for multi-turn agent workflows.

- **Dual-Mode Design:** Two lossless prompt layouts—Mode A (Authoritative Tail) and Mode B (Lazy Tail with line diffs and a bounded diff chain, `max_diffs_before_rebase` = 10)—with scenario-based recommendations derived from simulation (Table 7).

- **Complexity Reduction:** A derivation showing that file tokens per prompt fall from $\mathcal{O}(P \cdot M \cdot N)$ to $\mathcal{O}(M \cdot N + P \cdot M \cdot \sigma)$, and cumulative prompt tokens from $\mathcal{O}(P^2(C_d+MN))$ to $\mathcal{O}(P^2(C_d+M\sigma)+P\cdot MN)$: still quadratic in P, but with a coefficient roughly 25–30× smaller.
- **Two Deployment Topologies:** Execution either on the client side (IDE extensions, agent CLI) or unilaterally as a transparent server-side inference gateway filter; either topology can run either mode.
- **Simulated Validation:** A reference implementation evaluated on 20,000 scripted agents (824,728 turns) over four real repositories and on a 16-GPU H100 serving simulator. Mode A: 88.7% less KV-cache per agent than append-only and 3.57× more agent turns per hour under load. Mode B: 84.8% less KV-cache and 88.1% fewer computed prompt tokens with a warm prefix cache. Every session recovers byte-exact. A single pilot session on DeepSeek V4 (Mode A) showed a 57.7% prompt-token reduction at turn 4.

2 Background and Motivation

2.1 The Memory Wall in LLM Serving

During transformer autoregressive generation, keys and values computed for prior tokens are stored in the KV-cache to avoid recomputation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

For a model with L layers, H_{kv} attention heads, and hidden dimension d_h, the physical memory consumed by a sequence of length S at precision b bytes is:

$$\text{Mem}_{\text{KV}} = 2 \times L \times H_{kv} \times d_h \times b \times S \tag{2}$$

For a modern 70B parameter dense model or mixture-of-experts (MoE) architecture (e.g., DeepSeek, Llama-3-70B) utilizing 80 layers and Grouped-Query Attention (GQA) with FP8 precision ($b = 1$), every token requires approximately 160 KB of active KV-cache. A conventional 100,000-token prompt locks ≈ 16 GB of high-speed VRAM per user. In multi-tenant environments, a cluster of 80GB GPUs quickly saturates its memory capacity, reducing batch concurrency to single digits.

2.2 The Failure of Conventional Mitigations

Existing optimizations fail to fully resolve multi-turn agentic bloat:

- **Prefix Caching:** While prompt caching saves compute FLOPs by matching static prefixes, it *does not reduce VRAM consumption.* The cached states must remain pinned in HBM, consuming identical memory. Furthermore, when agents modify files mid-session, prefix matching is often partially invalidated.
- **Context Window Truncation (Sliding Window):** Dropping early messages destroys conversational continuity and breaks repository awareness.
- **Iterative Summarization:** Summarizing code history via LLM incurs additional API costs, introduces latency, and frequently drops critical structural details (variable names, types, line numbers).

2.3 Input-Output Asymmetry and Amdahl's Leverage in Agentic Workloads

Autonomous agentic coding workflows exhibit an extreme asymmetry between input ingestion and output generation. In a typical multi-file refactoring session spanning P turns, each turn loads M files averaging N tokens alongside dialogue history, while producing only a targeted diff or edit of $D_k \ll M \cdot N$ output tokens. Empirically, the ratio of ingested context tokens to generated output tokens frequently exceeds 300 : 1 to 400 : 1 in repository-level coding sessions.

This asymmetry reframes the optimization priority under Amdahl's Law [5]. Let f represent the fraction of end-to-end latency attributable to input prefill and KV-cache management versus output decoding. In conventional agentic workloads:

$$f = \frac{T_{\text{prefill}} + T_{\text{KV}}}{T_{\text{prefill}} + T_{\text{KV}} + T_{\text{decode}}} \tag{3}$$

where T_{prefill} scales with total input sequence length S, T_{KV} scales with memory saturation and cache thrashing, and T_{decode} scales linearly with generated token count G. A high token ratio S/G does not by itself imply a high time fraction f, because prefill processes tokens in parallel while decode is sequential; in our serving simulation, prefill occupied 29–93% of GPU time depending on load, while the ingested-to-generated token ratio of append-only agents was 876:1. Amdahl's Law bounds the speedup $\mathcal{S}$ from optimizing exclusively the decode path, which accounts for a fraction $1 - f$ of latency, by:

$$\mathcal{S}_{\text{decode-only}} \leq \frac{1}{f} \tag{4}$$

so the more input-dominated the workload, the less decode-only optimizations (speculative decoding, KV-quantization) can help. By contrast, reducing input redundancy—the core mechanism of HashedContext—directly addresses the dominant f term, yielding substantially higher system-level returns. Prior literature's focus on accelerating autoregressive decoding leaves the primary latency, token cost, and memory saturation bottleneck in agentic systems—repetitive prompt prefill and redundant KV-cache residency—largely unaddressed.

2.4 In-Situ Proactive Deduplication versus Post-Hoc Reactive Compaction

The preceding analysis establishes input redundancy as the dominant performance bottleneck in agentic coding systems. We now examine the structural divergence between reactive post-hoc compaction strategies (recursive LLM summarization, sliding-window truncation, rolling buffers) and HashedContext's proactive in-situ deduplication across three formal dimensions.

Secondary Computational Overhead

Existing approaches to context compression invoke auxiliary LLM calls to summarize or condense historical context after redundancy has already been serialized. Let T_{compact} denote total session wall time under a post-hoc strategy:

$$T_{\text{compact}} = T_{\text{session}} + \sum_{i=1}^{P} \Delta T_{\text{aux}}^{(i)} \tag{5}$$

where each $\Delta T_{\text{aux}}^{(i)}$ represents the latency of an auxiliary prefill-decode cycle for compaction at turn i. These cycles incur both additional FLOPs (forward passes over the accumulated context) and API-level token costs, which can amplify total inference expenditure by 2–5× over the base session [8]. In our simulated fleet, where compaction summarizes to 30% of a 128K-token threshold, summarization calls accounted for 11% of the compacting baseline's tokens; that baseline still cost less than plain append-only, but 3.5× more than HashedContext. Furthermore, recursive summarization requires the model to attend to the *entire* historical context during each compaction pass, exacerbating the very memory saturation it purports to alleviate.

By contrast, HashedContext operates at the transmission boundary on the CPU, computing a SHA-256 hash of each file observation prior to serialization. No auxiliary model invocation is required: $\Delta T_{\text{aux}} = 0$ for all turns. The marginal cost per file reduces to a single hash computation (≈ 1.8 ms per MB measured on one x86-64 core without SHA extensions, i.e., about 0.55 GB/s; about 0.12 ms for a typical turn's 78 KB of files), which is negligible relative to LLM prefill latency.

Syntactic Entropy Loss in Code Contexts

Natural language discourse tolerates semantic summarization: a human reader can follow a compressed narrative that omits exact word choices while preserving intended meaning. Formal software source trees, however, require exact byte-level AST reproducibility. Let $\mathcal{T}_{\text{code}}$ denote the ground-truth abstract syntax tree of a source file, and $\mathcal{S}_{\text{summary}}$ denote the summary produced by a reactive compaction pass. The conditional syntactic entropy introduced by summarization is:

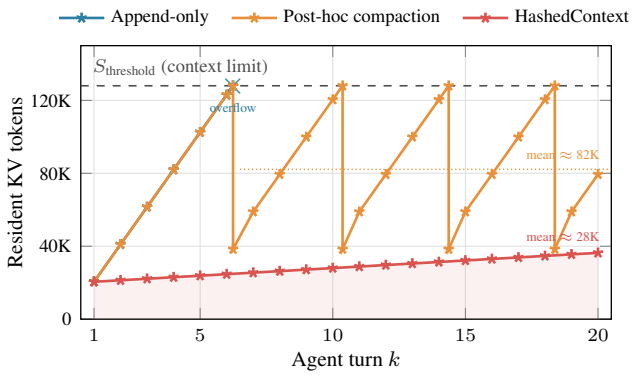


Figure 2: Resident KV-cache over a session (analytical model, M=5, N=4K, σ=67, C_d=500, compaction to 30% of a 128K threshold). Append-only context overflows by turn 7; post-hoc compaction keeps the cache in a sawtooth near the limit; HashedContext grows by $C_d + M\sigma$ per turn, so its dwell integral $\mathcal{M}_{\text{dwell}}$ (shaded) stays roughly 2.9× lower.

$$\mathcal{H}(\mathcal{T}_{\text{code}} \mid \mathcal{S}_{\text{summary}}) = -\sum_{t \in \mathcal{T}_{\text{code}}} P(t \mid \mathcal{S}_{\text{summary}}) \log P(t \mid \mathcal{S}_{\text{summary}}) > 0 \tag{6}$$

This non-zero entropy captures the probability that structural elements—variable declarations, type annotations, import paths, line-level diffs—are irrecoverably lost or hallucinated during condensation. In practice, recursive summarization of code histories has been shown to induce downstream compilation regressions and test failures, as the model operates on an imprecise internal reconstruction rather than the authoritative source [9].

HashedContext preserves lossless byte identity: because file contents are transmitted once and referenced via cryptographic stub thereafter, the LLM's working state always reflects the exact authoritative source. Under this regime, $\mathcal{H}(\mathcal{T}_{\text{code}} \mid \mathcal{S}_{\text{stub}}) = 0$, as the hash stub deterministically resolves to the unmodified original without information loss.

Cumulative VRAM Dwell Time

A critical yet underexamined property of long-running agent sessions is the cumulative memory allocation over the session lifecycle. The peak memory integral captures the total VRAM occupancy integrated across time:

$$\mathcal{M}_{\text{dwell}} = \int_0^{T_{\text{session}}} \text{Mem}_{\text{KV}}(t)\, dt \tag{7}$$

Under post-hoc compaction, $\text{Mem}_{\text{KV}}(t)$ follows a sawtooth profile: each turn appends $M \times N$ new tokens (and their KV vectors) to the cache, saturating toward the threshold $S_{\text{threshold}}$ before a compaction event releases

a fraction of the allocated memory. For the majority of interaction turns, however, $\mathrm{Mem}_{\mathrm{KV}}(t)$ dwells at near-saturation levels:

$$\frac{1}{T_{\text{session}}} \int_0^{T_{\text{session}}} \mathrm{Mem}_{\mathrm{KV}}(t)\, dt \approx S_{\text{threshold}} - \epsilon \tag{8}$$

where ϵ is the small transient reduction achieved by each compaction pass. This sustained pressure constrains GPU batch concurrency and forces frequent cache offloading to host memory, degrading tail latency. Figure 2 plots the resulting memory profiles under the cost model of Section 4.

HashedContext limits $\mathrm{Mem}_{\mathrm{KV}}(t)$ to one full copy of the live working set $\mathcal{W}(t)$—the files actively observed or modified in the current turn—plus dialogue history and one fixed-size stub per historical file observation. Each historical copy shrinks from $\mathcal{O}(N)$ to $\mathcal{O}(\sigma)$ tokens. At turn k, the instantaneous footprint satisfies:

$$\mathrm{Mem}_{\mathrm{KV}}(k) \le \kappa\Big(|\mathcal{W}(k)| \cdot N + k \cdot C_d + (k-1) \cdot M\sigma\Big) \tag{9}$$

where κ is the KV-cache size per token (163,840 bytes for the model of Section 5). The footprint still grows linearly with k, but by $C_d + M\sigma$ per turn rather than $C_d + MN$. Across our simulated fleet, the mean KV-cache needed per request was 3.5 GB under HashedContext versus 31.0 GB append-only and 11.3 GB with compaction.

In summary, while recursive summarization and sliding-window truncation provide tractable heuristics for open-ended natural language dialogue, deterministic software engineering workflows—where exact byte-level reproducibility, bounded VRAM occupancy, and zero auxiliary compute overhead are required—necessitate the in-situ content-addressed deduplication that HashedContext provides.

3 The HashedContext Architecture

HashedContext operates on a foundational principle: *The conversation history of an agent should serve as a state transition log, not a verbatim filesystem archive.* Figure 1 gives an overview of the components described below. Sections 3.5 and 3.6 apply to both modes; the tail-rebasing layout and Algorithm 1 describe Mode A, and Section 3.7 introduces Mode B and the dual-mode design.

3.1 Cryptographic Hash Stubs

Instead of re-emitting large file bodies into prompt messages, HashedContext abstracts unchanged files into standardized, compact textual stubs termed `HashStubs`:

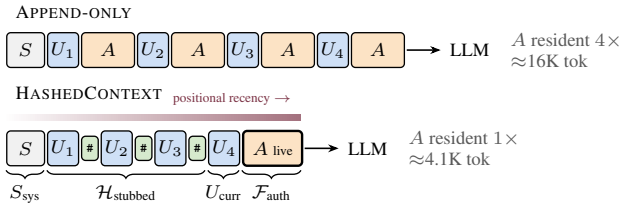


Figure 3: Prompt assembly at turn 4 for one file A. Append-only context interleaves four full copies of A with the dialogue. HashedContext keeps the dialogue U_k verbatim, replaces historical copies with stubs (`#`), and places the single authoritative copy last, where positional recency is strongest. Braces map segments to Eq. (10).

Listing 1: Standardized HashStub Structure

```
[HASH_STUB]
path: src/auth/session.ts
sha256: e4b2c91f7a62d88d3e91b40c21a7f05b
        8823190dfca91283c4b58e99128741aa
state: CACHED
[/HASH_STUB]
```

With the full 64-character digest on one line, this stub is 67 tokens under the `cl100k_base` and `o200k_base` tokenizers; across the real file paths in our simulated fleet the mean was $\sigma = 67.8$ tokens (range 53–88), of which the hex digest alone accounts for about 36. A compact variant that keeps 16 hex characters (`[#sha:<16 hex> path]`) averages about 20 tokens. Either form uniquely references the exact byte content of the file through the vault. We use $\sigma \approx 67$ throughout.

3.2 Attention-Anchored Authoritative Rebasing

Transformers exhibit strong positional recency bias: tokens placed near the end of the context window receive higher attention allocation than tokens situated in the middle of long prompts.

HashedContext exploits this inductive bias by maintaining exactly **one authoritative, fully materialized copy** of each referenced file, anchored strictly at the tail of the prompt assembly immediately preceding the model's generation head:

$$P_{\text{assembled}} = S_{\text{sys}} \circ \mathcal{H}_{\text{stubbed}} \circ U_{\text{curr}} \circ \mathcal{F}_{\text{authoritative}} \tag{10}$$

where S_{sys} is the system instruction, $\mathcal{H}_{\text{stubbed}}$ represents historical turns with files replaced by stubs, U_{curr} is the active user turn, and $\mathcal{F}_{\text{authoritative}}$ is the current canonical file state. Figure 3 contrasts this layout with conventional append-only assembly.

This achieves two objectives:

1. **Single Source of Truth:** The model cannot attend to obsolete syntax from earlier turns, eliminating multi-version hallucination regressions.

2. **Minimal Attention Overhead:** The historical transcript remains lightweight, allowing the attention mechanism to focus its parameters over the actual change boundaries.

Self-Attesting Live Payloads. When the authoritative file state $\mathcal{F}_{\text{authoritative}}$ is appended to the prompt tail, its structure directly encodes its own cryptographic digest within the metadata header:

$$\mathcal{F}_{\text{authoritative}} = \langle \text{path},\ \text{sha256}(\text{content}),\ \text{content} \rangle \quad (11)$$

where sha256(content) is the SHA-256 digest of the byte-exact file body computed at assembly time. This self-describing structure enables the model's self-attention heads to perform an $\mathcal{O}(1)$ cryptographic equivalence check entirely within the forward pass.

Systems consequence. When an agent or user queries whether a file has changed across turns, the model compares the live header hash—embedded in $\mathcal{F}_{\text{authoritative}}$ at the tail—directly against historical stub entries `[HASH_STUB]` within its attention matrix. No external retrieval tool, filesystem stat call, or secondary storage lookup is required. The equality check reduces to a token-level string comparison within the already-attended context window. This tells the model *which* file changed without any retrieval. It does not tell the model *what* changed, because the previous bytes are not in the prompt: in our mechanical needle test (Section 11), the changed file was identified in 100% of 300 cases from stubs alone, but the changed line in 0%. Recovering the change requires injecting the previous version via $\mathcal{R}(h)$ (Section 3.5), which raised line-level accuracy to 100% at a cost of 15% more prompt tokens.

3.3 Action Narrative Preservation and Semantic Losslessness

HashedContext enforces an event-sourced invariant that distinguishes two fundamental classes of content: **State Representations** and **Action Narratives**.

The State-Action Dichotomy. A *state representation* is a static file body—an immutable byte sequence that, when unchanged across turns, carries zero new information. HashedContext deduplicates these into hash stubs. An *action narrative* is the agent's reasoning trace: tool invocations, natural-language plans, edit diffs, test output, and validation logs. These constitute the permanent semantic ledger of the session. HashedContext never compresses, truncates, or stubs action narratives; they remain in raw, uncompressed textual form throughout the full interaction horizon.

Eliminating Boilerplate Attention Dilution. By stripping unchanged file bodies from the historical transcript while preserving every reasoning token and semantic diff, HashedContext focuses the model's finite attention budget on the actual mutations across turns. The consequence is twofold:

1. **Trajectory fidelity is preserved:** The full chronological chain of agent decisions, tool outputs, and validation results remains accessible for audit, rollback analysis, and task resumption.

2. **Semantic noise is suppressed:** Boilerplate file dumps that would otherwise dilute softmax probability mass across thousands of redundant tokens are replaced by compact stubs, forcing the attention mechanism to concentrate on the informative boundaries.

This design keeps every diff and reasoning step available to the model across the session. Whether reasoning accuracy is preserved in practice must be measured with a live LLM; Section 11 reports what information each prompt contains, and live-model evaluation is future work.

3.4 Formal Assembly Algorithm

Algorithm 1 details the prompt reconstruction process executed per interaction turn.

Algorithm 1 HashedContext Prompt Reconstruction

Require: Conversation history $\mathcal{M}$, Active Files $\mathcal{F}$, Cache $\mathcal{C}$, Vault $\mathcal{V}$
Ensure: Compact, Authoritative Prompt P

$\mathcal{H}_{\text{compact}} \leftarrow [\,]$
for each message $m \in \mathcal{M}_{0\ldots|\mathcal{M}|-2}$ **do**
  $m' \leftarrow m$
  for each file block $f \in m.\text{files}$ **do**
    $h \leftarrow \text{SHA256}(f.\text{content})$
    $\mathcal{V}[h] \leftarrow f.\text{content}$ ▷ Vault historical payload
    $\mathcal{C}[f.\text{path}] \leftarrow h$
    $m' \leftarrow \text{ReplaceWithStub}(m', f.\text{path}, h)$
  end for
  $\mathcal{H}_{\text{compact}}.\text{append}(m')$
end for
$P_{\text{tail}} \leftarrow [\,]$
for each file $f \in \mathcal{F}_{\text{active}}$ **do**
  $P_{\text{tail}}.\text{append}(\text{FormatLiveState}(f))$
end for
$P \leftarrow \mathcal{H}_{\text{compact}} \circ \mathcal{M}_{\text{last}} \circ P_{\text{tail}}$
return P
▷ On-demand: $\mathcal{R}(h) \rightarrow \mathcal{V}[h]$ ▷ See §3.5

3.5 Hierarchical Context Tiering and On-Demand State Resolution

HashedContext partitions context storage into two hierarchical tiers to decouple active inference memory from historical archive bulk:

1. **Primary In-Context Working Set:** Active live files and recent instructions are maintained resident in the generation context within GPU KV-cache. This tier corresponds to the prompt tail $\mathcal{F}_{\text{authoritative}}$ in

Equation (1) and encompasses the current file system snapshot immediately visible to the attention mechanism.

2. **Secondary Content-Addressed Vault ($\mathcal{V}$):** All historical file payloads are persisted in a local content-addressed store, the *HashVault*, indexed deterministically by their SHA-256 digest. Each vault entry is a mapping $\mathcal{V}[\text{hash}] \mapsto \text{byte_content}$, enabling instant retrieval without re-reading from the filesystem or re-requesting from the agent.

On-Demand Historical Resolution. In nominal operation, only the stub representation of historical files enters the prompt assembly. However, when an agentic task requires cross-turn analysis—comparing the current file state against a version observed k turns ago, or inspecting a diff across historical variants—the system resolves the referenced `[HASH_STUB]` from the local HashVault. Formally, given a stub referencing hash h, the resolution function $\mathcal{R}$ retrieves the verified byte content:

$$\mathcal{R}(h) = \begin{cases} \mathcal{V}[h] & \text{if } h \in \text{dom}(\mathcal{V}) \\ \emptyset & \text{otherwise (cache miss)} \end{cases} \tag{12}$$

The resolved content is injected into the prompt payload for that single evaluation cycle only; it does not persist across subsequent turns, thereby avoiding permanent KV-cache pressure. This mechanism guarantees **lossless historical state preservation under negligible collision probability**: full historical state is always recoverable on demand via SHA-256 content addressing ($\mathcal{O}(2^{-256})$ collision bound) with sub-millisecond local disk latency (median 32 μs, p99 0.52 ms measured with a warm page cache), without permanently penalizing active GPU VRAM.

Zero Information Loss Guarantee. Because every historical file version is content-addressed by its SHA-256 digest and stored in $\mathcal{V}$, the system guarantees lossless historical state recovery under a collision probability bounded by $\mathcal{O}(2^{-256})$—effectively negligible for any practical session horizon. For any interaction turn k, the complete state $\mathcal{S}_k$—including every file version ever observed—can be reconstructed by iterating the conversation log $\mathcal{M}$ and resolving each stub encountered:

$$\forall k \in [1, P],\ \mathcal{S}_k = \bigcup_{m \in \mathcal{M}_{0..k}} \bigcup_{\text{stub } s \in m} \mathcal{R}(s.\text{hash}) \tag{13}$$

This stands in contrast to lossy compression techniques (sliding windows, summarization) that irrevocably discard information. The cost of this guarantee is bounded storage on local disk—0.36 MB of unique content per session on average in our simulated fleet—while GPU memory consumption remains strictly proportional to the working set size, not the session horizon.

3.6 Pass-by-Reference Context Semantics

Conventional multi-turn prompt construction follows the semantics of **Pass-by-Value** in programming language theory: every turn produces a fully materialized copy of the entire conversation history, including all previously transmitted file bodies. In a system with M files averaging N tokens over P turns, this induces $\mathcal{O}(M \times N \times P)$ redundant memory and network duplication—each turn performs a deep copy of the cumulative state. Formally, the conventional append-only log at turn k contains $\sum_{i=1}^{k} \left(C_d + \sum_{j=1}^{M} [\text{file}_j^{(i)}]\right)$ tokens, where each file body is independently serialized regardless of immutability.

HashedContext replaces this with a **Pass-by-Reference** protocol. The `[HASH_STUB]` acts as an immutable 32-byte cryptographic pointer in the conversational history, analogous to a read-only reference in a garbage-collected runtime. The tail anchor $\mathcal{F}_{\text{authoritative}}$ (Equation 1) functions as the single dereferenced working instance—the authoritative memory location that all prior stubs conceptually reference. Under this model:

- **Identity, Not Equality:** Each stub carries a SHA-256 digest that uniquely identifies the referenced content. Two stubs with identical digests are guaranteed (under standard cryptographic assumptions) to reference identical byte sequences, enabling $\mathcal{O}(k)$ stub-to-stub comparison rather than $\mathcal{O}(N)$ byte-level comparison.
- **No Deep Copy on Read:** Historical stubs are *not* expanded during nominal inference. The model attends to the stub token sequence (≈67 tokens) rather than the materialized file body, eliminating the $\mathcal{O}(M \times N \times P)$ deep-copy cost that characterizes Pass-by-Value architectures.
- **Lazy Materialization:** Full file content is materialized only at two points: (a) in the authoritative tail $\mathcal{F}_{\text{authoritative}}$ for the current turn, and (b) on-demand via $\mathcal{R}(h)$ for cross-turn analysis (Section 3.5). This mirrors call-by-need evaluation in functional languages.

The semantic shift from Pass-by-Value to Pass-by-Reference is the foundational abstraction that enables HashedContext to decouple session horizon from KV-cache footprint. Without this re-framing, the $\mathcal{O}(M \times N \times P)$ scaling is an inescapable property of the conventional append-only log semantics.

3.7 Dual-Mode Architecture

Mode A rewrites history every turn: the previous live copy becomes a stub and a new live copy is placed at the tail. This keeps one complete current copy of every file in view, but it also means the tail's KV-cache cannot be reused on the next turn. Mode B makes the opposite choice. It

never rewrites history, so each prompt is a strict prefix of the next and a prefix cache can reuse almost all of it.

Mode A: Authoritative Tail. The layout of Equation (10) and Algorithm 1: historical copies become stubs, and the files observed in the current turn are materialized once at the tail with a self-attesting digest.

Mode B: Lazy Tail. Each file observation is appended to the history in one of three forms (Algorithm 2):

- *First sight:* the full body, once, with its digest.
- *Unchanged since last shown:* a stub whose state line reads "Turn X content is still current" (≈73 tokens).
- *Changed since last shown:* only the changed lines relative to the last version the model saw, as a line diff with the base and new digests (median 104 tokens).

The current state of a file is its last full body plus every diff since. To bound how many diffs the model must apply, a file that has accumulated `max_diffs_before_rebase` diffs since its last full body (default 10) is sent in full on its next change (a *rebase*). Rebasing costs memory: without it, diff chains in our simulated runaway loops reached 99. The diff is line-based, not syntax-aware.

Algorithm 2 Mode B (Lazy Tail) observation rendering

Require: Turn k, observations $\mathcal{F}_k$, seen map $\mathcal{S}$, chain counts c, bound $D_{\max}$

for each file $f \in \mathcal{F}_k$ **do**
  $h \leftarrow \text{SHA256}(f.\text{content})$
  if $f.\text{path} \notin \mathcal{S}$ **or** $(\mathcal{S}[f.\text{path}].h \neq h$ **and** $c[f.\text{path}] \geq D_{\max})$ **then**
    emit $\text{Full}(f, h)$; $c[f.\text{path}] \leftarrow 0$
  else if $\mathcal{S}[f.\text{path}].h = h$ **then**
    emit $\text{Stub}(f.\text{path}, h, \mathcal{S}[f.\text{path}].\text{turn})$
  else
    emit $\text{Diff}(\mathcal{S}[f.\text{path}].\text{content} \rightarrow f.\text{content})$; $c[f.\text{path}] \leftarrow c[f.\text{path}] + 1$
  end if
  $\mathcal{S}[f.\text{path}] \leftarrow (f.\text{content}, h, k)$ **unless** a stub was emitted
end for
append U_k and the emitted blocks to the history (never rewritten)

Properties. Both modes are lossless: every Mode B diff is verified by applying it to the previous version and checking the SHA-256 of the result, and every file can be rebuilt from the prompt text alone. Mode B shows the model exactly which lines changed without $\mathcal{R}(h)$; Mode A shows the complete current file without requiring any reconstruction.

Choosing a mode. The right mode depends on the serving scenario, summarized in Table 7 and measured in Section 11.2. In short, Mode B wins when the prefix cache stays warm (sticky routing, light load, or APIs that discount cached tokens). Mode A wins under heavy load, without a prefix cache, and when the model should always see one complete current copy, for example in long edit loops or accuracy-critical work. Mode B's diff chains are its main risk. In a live test (Section 11, Table 8), one model answered 90.0% of current-content questions correctly from a 10-diff chain and 93.3% with $N = 5$, versus 93.3% for Mode A; at 30 cases these differences are within noise, so for accuracy-critical work we recommend $N = 5$, which costs about 16% more prompt tokens in that test. The "Regime A/B" terminology of Section 4.3 refers to whether a prefix cache is available, which is one of the inputs to this choice.

4 Mathematical Formulation

Consider a multi-turn agent interaction spanning P discrete turns, interacting over a working set of M files, where the average file token size is N.

4.1 Conventional Append-Only Cost

In standard agent architectures, every turn accumulates history while re-requesting and re-reading active files. The total cumulative prompt tokens transmitted and ingested over P turns is:

$$T_{\text{standard}}(P) = \sum_{k=1}^{P} \left(k \cdot C_{\text{dialogue}} + k \cdot \sum_{i=1}^{M} N_i \right) \tag{14}$$

Assuming fixed average dialogue tokens C_d and identical average file size N:

$$\begin{aligned} T_{\text{standard}}(P) &\approx \frac{P(P+1)}{2} \cdot (C_d + M \cdot N) \\ &\in \mathcal{O}(P^2 \cdot M \cdot N) \end{aligned} \tag{15}$$

The token consumption scales quadratically with respect to interaction depth.

4.2 HashedContext Cost

Under HashedContext, historical file occurrences are collapsed into a constant stub size σ ($\sigma \approx 67$ tokens, Section 11), and the complete file body is materialized only once at the tail:

$$T_{\text{HashedContext}}(P) = \sum_{k=1}^{P} (k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k) \tag{16}$$

where D_k represents the modified delta token volume at turn k. In sparse editing workflows where $D_k \ll M \cdot N$:

$$\begin{aligned} T_{\text{HashedContext}}(P) \approx \frac{P(P+1)}{2} C_d + \frac{P(P-1)}{2} M\sigma \\ + P(M \cdot N) \end{aligned} \tag{17}$$

Both totals are quadratic in P: HashedContext lowers the quadratic coefficient from $(C_d + MN)/2$ to $(C_d + M\sigma)/2$

and adds a linear $P \cdot MN$ term for the live tail. The asymptotic saving ratio $R(P)$ for file context as P increases is:

$$\lim_{P \to \infty} R(P) = 1 - \frac{\sigma}{N} \tag{18}$$

For a typical source file of $N = 4,000$ tokens and a stub size of $\sigma = 67$ tokens:

$$R_\infty = 1 - \frac{67}{4,000} = 98.3\% \tag{19}$$

Including dialogue, the total saving tends to $1 - (C_d + M\sigma)/(C_d + MN) = 1 - 835/20{,}500 = 95.9\%$ for $M = 5$, $C_d = 500$. In our simulated fleet (Section 11), the measured reduction over all 824,728 turns was 88.7% of prompt tokens and 93.2% of file tokens; for the five-file profile the median per-turn reduction rose from 72.8% at turn 4 to 94.1% at turn 30.

4.3 Dual-Regime Operational Robustness

HashedContext delivers performance guarantees across two fundamentally distinct serving environments, analyzed separately below.

4.3.1 Regime A: Prefix-Cached Serving

In environments where a shared prefix cache is available (e.g., vLLM's automatic prefix caching, SGLang's RadixAttention), an append-only prompt at turn k is exactly the turn $k-1$ prompt plus new text. Edits append new file copies rather than rewriting history, so append-only prompts already reuse nearly their whole prefix. In our serving simulation with one or two agents per GPU, append-only prompts achieved 88.6–90.6% prefix-cache hit rates, and the pilot telemetry in Table 9 shows 97.8%.

Under HashedContext, the prefix shared across consecutive turns is:

$$\text{Prefix}_{\text{shared}}(k) = \sum_{i=1}^{k-1} \left(C_d + M \cdot \sigma\right) \tag{20}$$

Each turn, the previous live tail is replaced by a stub, so the tail's KV must be recomputed. HashedContext therefore reaches a *lower* hit rate (65.0–69.6% in the same simulation), and with an ideal per-session cache that never evicts, it computes 3.6% more prompt tokens than append-only.

The benefit in this regime is KV residency, not cache reuse. Across the simulated fleet, the mean KV-cache needed per request falls from 31.0 GB (append-only) and 11.3 GB (with compaction) to 3.5 GB, a reduction of 88.7% and 69% respectively. Under concurrency the append-only cache cannot stay resident: on a simulated 16-GPU cluster serving 512 agents, its hit rate fell to 0.3%, and HashedContext computed 60% fewer prompt tokens per turn. Sessions share only the system prompt unless their dialogue is identical, and we observed no meaningful cross-session reuse. The analysis above describes Mode A. Mode B (Section 3.7) keeps history append-only, reaching a 96.8% hit rate in the same accounting and computing 88.1% fewer prompt tokens than append-only, at the cost of a larger resident prompt (4.71 GB versus 3.50 GB of KV per request).

4.3.2 Regime B: Uncached Serving

In production deployments, distributed load balancing may route turn k and turn $k+1$ to different physical worker nodes lacking a shared prefix cache. Similarly, cold-start scenarios (new model deployment, cache eviction under memory pressure) begin with an empty prefix cache. Under these zero-cache conditions, conventional architectures regress to full quadratic prefill: each turn recomputes KV vectors for the entire accumulated history. For a deep turn k, this requires recomputing $\Theta(k \cdot M \cdot N)$ attention operations.

HashedContext protects Time-to-First-Token (TTFT) even under zero-cache conditions. In Regime B, the stub-compressed history requires prefill over:

$$S_{\text{stubbed}}(k) = k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k \tag{21}$$

compared to the conventional:

$$S_{\text{conventional}}(k) = k \cdot C_d + k \cdot M \cdot N \tag{22}$$

The prefill FLOPs ratio is:

$$\begin{aligned} \frac{\text{FLOPs}_{\text{HC}}(k)}{\text{FLOPs}_{\text{conv}}(k)} &\approx \frac{k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N}{k \cdot C_d + k \cdot M \cdot N} \\ &\xrightarrow{k \to \infty} \frac{C_d + M\sigma}{C_d + MN} \end{aligned} \tag{23}$$

counting linear (projection) FLOPs, which scale with sequence length. Dialogue does not shrink, so the limit is $835/20{,}500 \approx 1/24.6$ for $M = 5$, $N = 4{,}000$, $C_d = 500$, $\sigma = 67$, not σ/N. Concretely, at turn $k = 10$:

- Conventional prefill: $S_{\text{conv}} \approx 10 \times C_d + 200{,}000$ tokens.
- HashedContext prefill: $S_{\text{HC}} \approx 10 \times C_d + 23{,}015$ tokens.

HashedContext reduces redundant prefill from $\Theta(k \cdot M \cdot N)$ to $\mathcal{O}(M \cdot N + k \cdot M \cdot \sigma)$ tokens per turn, which matters most when cache locality cannot be guaranteed. With no prefix cache, it computed 88.7% fewer prompt tokens than append-only across our simulated fleet.

5 Dual-Phase Latency and Memory-Bandwidth Dynamics

Transformer-based language model inference is fundamentally bifurcated into two distinct computational phases: the *prefill* (or encoding) phase and the *autoregressive decoding* phase. Each phase exhibits a distinct performance

profile governed by different hardware bottlenecks. In this section, we formally decompose these phases, quantify the physical memory-bus traffic induced during decoding, and demonstrate how HashedContext's sequence-length reduction directly relieves the dominant bandwidth bottleneck.

5.1 Prefill Phase: Compute-Bound Encoding

During prefill, the transformer processes all S input tokens in a single forward pass, computing the full $S \times S$ attention matrix. The time-to-first-token (TTFT) is dominated by matrix–matrix multiplication operations that saturate GPU tensor cores:

$$T_{\text{prefill}}(S) = \frac{4 \cdot S \cdot d_{\text{model}} \cdot L}{\text{FLOPS}_{\text{peak}}} + \frac{S^2 \cdot L}{\text{FLOPS}_{\text{attn}}} \tag{24}$$

where d_{model} is the model dimension, L is the number of layers, $\text{FLOPS}_{\text{peak}}$ is the peak tensor-core throughput, and $\text{FLOPS}_{\text{attn}}$ is the effective attention computation throughput. The first term represents the $\mathcal{O}(S)$ projection FLOPs; the second term represents the $\mathcal{O}(S^2)$ attention FLOPs that dominate at long sequence lengths.

Under conventional append-only architectures, the prefill sequence length at turn k is $S_{\text{conv}}(k) = k \cdot C_d + k \cdot M \cdot N$, which grows linearly with turn count. This produces a TTFT that scales as $\Theta(k^2)$ in the attention term, causing progressively worsening response latency on deep turns. Under HashedContext, the prefill length is $S_{\text{HC}}(k) = k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k$, whose dominant term at turn $k \gg 1$ is $k \cdot C_d + k \cdot M \cdot \sigma$. The sequence-length ratio $S_{\text{conv}}/S_{\text{HC}}$ depends on depth: $7.3\times$ at $k = 10$, approaching $(C_d + MN)/(C_d + M\sigma) \approx 24.6\times$ for large k ($M = 5$, $N = 4{,}000$, $C_d = 500$, $\sigma = 67$). The projection term shrinks by this ratio and the S^2 attention term by up to its square. Both architectures still grow with k; HashedContext grows far more slowly.

5.2 Decode Phase: Memory-Bandwidth-Bound Generation

Following prefill, the model enters autoregressive decoding, generating one token at a time. At each decoding step i, the model computes attention between the i-th query and all $S + i - 1$ cached Key-Value vectors. Crucially, this operation is not compute-bound but **memory-bandwidth bound**: the dominant cost is reading the entire KV-cache from HBM into on-chip SRAM for each generated token.

The total latency for K generated tokens is:

$$T_{\text{total}} = T_{\text{prefill}}(S) + \sum_{i=1}^{K} T_{\text{decode}}(S + i) \tag{25}$$

Each decode step's latency is determined by the time required to transfer KV vectors across the HBM3 memory bus:

$$T_{\text{decode}}(S_{\text{active}}) = \frac{\text{Traffic}_{\text{step}}(S_{\text{active}})}{B_{\text{HBM}}} \tag{26}$$

where B_{HBM} is the effective HBM bandwidth (e.g., $\approx$ 3.35 TB/s for NVIDIA H100 SXM) and $\text{Traffic}_{\text{step}}$ is the total data movement per step.

5.3 Memory-Bus Traffic Per Decoding Step

The physical data movement per autoregressive decoding step is a function of the model architecture and the active sequence length. For each generated token, every transformer layer must read all Key and Value vectors for all preceding tokens from HBM into SRAM:

$$\text{Traffic}_{\text{step}}(S_{\text{active}}) = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot S_{\text{active}} \tag{27}$$

where:

- L = number of transformer layers,
- H_{kv} = number of KV-heads in Grouped-Query Attention,
- d_h = dimension per head,
- b = bytes per element (e.g., $b = 1$ for FP8),
- S_{active} = number of cached tokens attended to at the current step.

The factor of 2 accounts for both Key and Value tensors being loaded. For a modern 70B-class MoE model with $L = 80$ layers, $H_{kv} = 8$ KV-heads (GQA ratio 8:64), $d_h = 128$, and $b = 1$ (FP8):

$$\begin{aligned} \text{Traffic}_{\text{step}}(S_{\text{active}}) &= 2 \cdot 80 \cdot 8 \cdot 128 \cdot 1 \cdot S_{\text{active}} \\ &= 163{,}840 \cdot S_{\text{active}} \text{ bytes} \end{aligned} \tag{28}$$

At a sequence length of $S_{\text{active}} = 100{,}000$ tokens, each decode step requires moving 16.4 GB across the HBM bus. With H100 HBM bandwidth of 3.35 TB/s, this yields:

$$T_{\text{decode}}(100{,}000) = \frac{16.4 \times 10^9}{3.35 \times 10^{12}} \approx 4.9 \text{ ms per step} \tag{29}$$

At $S_{\text{active}} = 20{,}000$ tokens, the KV read drops to 0.98 ms per step, a $5\times$ reduction in KV traffic. Every decode step must also read the model weights, however. With $M_{\text{weights}} = 30\,\text{GB}$, a single request's step time is $(30 + 16.4)\,\text{GB}/3.35\,\text{TB/s} = 13.85\,\text{ms}$ at 100K tokens and 9.93 ms at 20K tokens, a $1.39\times$ improvement per request at batch size 1. The larger gain comes from batching, discussed below.

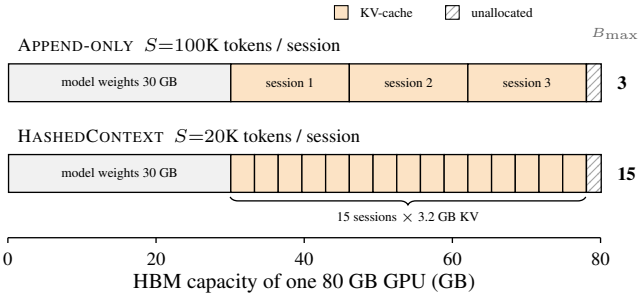


Figure 4: Batch concurrency on one 80 GB GPU (70B-class GQA model in FP8, ≈160 KB of KV per token). Shrinking each session's resident context from 100K to 20K tokens raises the number of concurrent sequences $B_{\max}$ from 3 to 15 (Eq. 32).

5.4 Sequence-Length Leverage on Throughput

The token generation throughput, measured in tokens per second, is:

$$\text{Throughput} = \frac{B}{T_{\text{step}}}, \qquad T_{\text{step}} = \frac{M_{\text{weights}} + \sum_{j=1}^{B} 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot S_j}{B_{\text{HBM}}} \tag{30}$$

where B is the number of sequences decoded together. The weights are read once per step for the whole batch, so throughput rises with batch size, and the batch size that fits in memory is set by the active sequence lengths S_j. Reducing S_{active} therefore increases generated tokens per second mainly by allowing larger batches. Figure 1 illustrates how HashedContext collapses the effective S_{active} by removing redundant historical context.

To quantify the end-to-end throughput improvement, consider the generated tokens per second over an entire session:

$$\text{TPT}_{\text{session}} = \frac{\sum_{k=1}^{P} G_k}{\sum_{k=1}^{P} T_{\text{prefill}}(S_k) + \sum_{k=1}^{P} \sum_{i=1}^{G_k} T_{\text{decode}}(S_k + i)} \tag{31}$$

where G_k is the number of generated tokens at turn k. At batch size 1, a 5× reduction in S_{active} gives only 1.39× faster decoding, because weight reads dominate. At full memory, three 100K-token sequences and fifteen 20K-token sequences both take 23.6 ms per step, so the same step time yields 5× more tokens. In our simulated 16-GPU cluster, generated tokens per second rose 2.5× (335 → 824) and agent turns per hour 3.6×.

5.5 Batch-Concurrency Amplification

In multi-tenant serving, the GPU memory capacity M_{GPU} constrains the maximum batch size:

$$B_{\max} = \left\lfloor \frac{M_{\text{GPU}} - M_{\text{model weights}}}{M_{\text{KV per sequence}}} \right\rfloor \tag{32}$$

where $M_{\text{KV per sequence}} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot S$. With $M_{\text{GPU}} = 80$ GB, model weight overhead of $M_{\text{weights}} \approx 30$ GB for a compressed 70B model, and per-sequence KV-cache of 16 GB (at $S = 100{,}000$), the maximum batch concurrency is:

$$B_{\max}(S = 100{,}000) = \left\lfloor \frac{80 - 30}{16} \right\rfloor = 3 \text{ concurrent sequences} \tag{33}$$

Under HashedContext with $S = 20{,}000$, per-sequence KV-cache is 3.2 GB, yielding:

$$B_{\max}(S = 20{,}000) = \left\lfloor \frac{80 - 30}{3.2} \right\rfloor = 15 \text{ concurrent sequences} \tag{34}$$

This represents a 5× improvement in batch concurrency in this analytical example (Figure 4). The 20K figure is close to what we measured: the mean HashedContext prompt across our simulated fleet was 21,349 tokens (3.5 GB of KV), versus 188,953 tokens append-only. In the simulated 16-GPU cluster, with the KV pool 99.7% full in both modes, KV per running request fell from 14.1 GB (append-only with compaction) to 3.55 GB, and the mean number of requests running at once rose from 3.5 to 14.1 (4.0×). In dollar terms, for an inference operator deploying a cluster of N_{GPU} GPUs, the effective throughput per dollar scales proportionally to $B_{\max}$, directly reducing the cost-per-token for agentic workloads.

6 Attention Entropy and Semantic Signal-to-Noise Ratio

A central but frequently overlooked consequence of context bloat in transformer-based agents is the degradation of attention allocation quality. As extraneous boilerplate tokens accumulate across turns, the softmax attention distribution becomes increasingly diffuse, forcing the model to distribute finite probability mass over tokens that carry zero semantic information about the current task. In this section, we formalize this degradation through the lens of attention entropy and introduce a contextual signal-to-noise ratio (SNR) that quantifies the information density of agentic prompts.

6.1 Softmax Attention and Context Dilution

Recall the standard transformer attention mechanism [3]. For a query vector q_i at position i and a sequence of key vectors $k_1, \ldots, k_S$, the attention probability distribution is:

$$\alpha_{ij} = \frac{\exp\left(q_i k_j^T / \sqrt{d_k}\right)}{\sum_{l=1}^{S} \exp\left(q_i k_l^T / \sqrt{d_k}\right)}, \quad \sum_{j=1}^{S} \alpha_{ij} = 1 \tag{35}$$

The entropy of this distribution quantifies how diffuse or concentrated the attention is:

$$\mathbb{H}(\alpha_i) = -\sum_{j=1}^{S} \alpha_{ij} \log \alpha_{ij} \tag{36}$$

When the context window contains S tokens, of which S_{signal} carry task-relevant information and S_{noise} are boilerplate or redundant, the attention entropy is maximized when probability mass is spread uniformly. Under the null hypothesis that the model allocates attention proportional to informativeness, the effective number of attended tokens is $S_{\text{eff}} = \exp(\mathbb{H}(\alpha_i))$. As S_{noise} grows, $\mathbb{H}(\alpha_i)$ increases, and α_{ij} over informative tokens decreases proportionally.

This phenomenon is empirically confirmed by the "lost-in-the-middle" literature: as context length grows and relevant information is positioned away from the prompt boundaries, model accuracy on retrieval tasks decays sharply. In agentic coding contexts, this translates to the model overlooking critical diffs, variable declarations, or API signatures buried under thousands of boilerplate file tokens.

6.2 Contextual Signal-to-Noise Ratio

We define the Contextual Signal-to-Noise Ratio ($\text{SNR}_{\text{context}}$) for a given prompt assembly P as:

$$\text{SNR}_{\text{context}}(P) = \frac{|\mathcal{T}_{\text{diff}}(P)| + |\mathcal{T}_{\text{authoritative}}(P)|}{|\mathcal{T}_{\text{boilerplate}}(P)| + |\mathcal{T}_{\text{redundant}}(P)|} \tag{37}$$

where:

- $\mathcal{T}_{\text{diff}}$ = tokens representing semantic mutations across turns (edit diffs, error messages, action narratives),
- $\mathcal{T}_{\text{authoritative}}$ = tokens in the single canonical file state (the authoritative tail),
- $\mathcal{T}_{\text{boilerplate}}$ = tokens from unchanged file bodies re-serialized into history,
- $\mathcal{T}_{\text{redundant}}$ = tokens from obsolete file versions that conflict with the current state.

In conventional append-only architectures at turn k, the number of noise tokens is:

$$|\mathcal{T}_{\text{boilerplate}}| + |\mathcal{T}_{\text{redundant}}| = k \cdot M \cdot N - \left(M \cdot N + \sum_{t=1}^{k-1} |\mathcal{T}_{\text{diff}}^{(t)}| \right) \tag{38}$$

while the signal tokens are:

$$|\mathcal{T}_{\text{diff}}| + |\mathcal{T}_{\text{authoritative}}| = M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}| \tag{39}$$

The conventional SNR at deep turn k therefore asymptotically approaches:

$$\lim_{k\to\infty} \text{SNR}_{\text{conv}}(k) = \frac{M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}|}{k \cdot M \cdot N - M \cdot N} \approx \frac{1}{k-1} \ll 1 \tag{40}$$

In deep conventional sessions, signal is therefore diluted harmonically, as $1/(k-1)$, by accumulated redundant copies.

6.3 HashedContext SNR Maximization

Under HashedContext, all historical file bodies are replaced by σ-token stubs ($\sigma \approx 67$), and the authoritative current state is materialized exactly once at the tail. The noise tokens collapse to:

$$|\mathcal{T}_{\text{boilerplate}}| + |\mathcal{T}_{\text{redundant}}| = (k-1) \cdot M \cdot \sigma \tag{41}$$

while signal tokens remain:

$$|\mathcal{T}_{\text{diff}}| + |\mathcal{T}_{\text{authoritative}}| = M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}| \tag{42}$$

The HashedContext SNR is therefore:

$$\text{SNR}_{\text{HC}}(k) = \frac{M \cdot N + \sum_{t=1}^{k} |\mathcal{T}_{\text{diff}}^{(t)}|}{(k-1) \cdot M \cdot \sigma} \tag{43}$$

As $k \to \infty$, this converges to:

$$\lim_{k\to\infty} \text{SNR}_{\text{HC}}(k) = \frac{|\overline{\mathcal{T}_{\text{diff}}}|}{M \cdot \sigma} \tag{44}$$

where $|\overline{\mathcal{T}_{\text{diff}}}|$ is the average diff token volume per turn. This limit is not necessarily large: with $|\overline{\mathcal{T}_{\text{diff}}}| = 150$, $M = 5$ and $\sigma = 67$, it is 0.45, so at depth the stubs themselves become the main noise term (a compact ≈20-token stub raises it to 1.5). The ratio $\text{SNR}_{\text{HC}}/\text{SNR}_{\text{conv}}$ scales as:

$$\frac{\text{SNR}_{\text{HC}}(k)}{\text{SNR}_{\text{conv}}(k)} \approx \frac{N}{\sigma} \approx 60\times \quad (N = 4{,}000,\ \sigma = 67) \tag{45}$$

This SNR is a token-count measure of how much of the prompt is redundant. It suggests that HashedContext leaves the model less redundant content to attend to, but it is not a measurement of attention itself; whether attention actually concentrates on the informative regions must be tested empirically.

6.4 Attention Entropy Reduction

We conjecture that the entropy of the attention distribution under HashedContext is lower than under conventional assembly for turns $k > 1$. Let $\mathbb{H}^{(k)}_{\text{conv}}$ and $\mathbb{H}^{(k)}_{\text{HC}}$ denote the average attention head entropy at turn k under conventional and HashedContext assembly, respectively. Because $\sigma \ll N$ and the stub token sequence is short and deterministic, we expect:

$$\Delta\mathbb{H}^{(k)} = \mathbb{H}^{(k)}_{\text{conv}} - \mathbb{H}^{(k)}_{\text{HC}} > 0, \quad \forall k > 1 \tag{46}$$

and that the entropy gap grows with k:

$$\frac{\partial}{\partial k}\Delta\mathbb{H}^{(k)} > 0 \tag{47}$$

These are hypotheses, not results: attention entropy depends on learned weights, not only on token counts, and we have not measured it. If they hold, diff detection, code reasoning and edit accuracy should degrade less with session depth under HashedContext than under append-only assembly. Testing this requires instrumenting a live model's attention or measuring task accuracy with a live LLM at increasing depth, which we leave to future work.

7 Inference Engine Topology: Prefix-Tree and Cluster Stability

Modern LLM serving engines rely on prefix-tree data structures to maximize KV-cache reuse across requests. In this section, we analyze how HashedContext's deterministic stub sequences interact with these caching topologies. The main effect is not higher cache reuse but smaller cached content: stubs keep each branch's interior nodes small.

7.1 Caching Topologies in Production Engines

Two dominant caching architectures have emerged in the LLM serving ecosystem:

RadixAttention (SGLang). SGLang [6] organizes KV-cache blocks in a radix tree (prefix tree) data structure. Each node in the tree represents a common prefix shared across multiple requests. When a new request arrives, SGLang traverses the radix tree to find the longest matching prefix, computes KV-cache only for the unmatched suffix, and inserts the new path into the tree. Cache eviction follows a least-recently-used (LRU) policy on tree nodes.

PagedAttention with Automatic Prefix Caching (vLLM). vLLM [1] implements PagedAttention, which partitions the KV-cache into fixed-size physical blocks. Its automatic prefix caching (APC) extension hashes block contents to detect shared prefixes across sequences: blocks with identical content hash are mapped to the same physical GPU memory page, enabling implicit prefix sharing without an explicit tree structure. A hash table maps content hashes to physical block addresses.

Both architectures exploit the observation that sequences with common prefixes produce identical KV vectors for those prefix tokens, enabling memory deduplication and computational savings.

7.2 The Prefix-Tree Fragmentation Problem

Despite their effectiveness, both RadixAttention and PagedAttention-APC suffer from a fundamental limitation: the prefix tree (or block-hash table) is sensitive to **intermediate text variations**. In agentic workflows, every unique file content across historical turns constitutes a unique intermediate sequence prefix. Concretely, consider a conventional agent session with $M = 3$ files over $P = 10$ turns. At turn k, the prompt includes k copies of each file—but each copy may differ if the file was edited in a prior turn. The resulting prefix tree exhibits:

- **Branching Across Trajectories:** Within a single append-only session, each turn extends the previous prompt, so consecutive turns share their whole prefix and do not fragment the tree. Branching occurs *between* trajectories: any two requests whose histories differ, even by one token of file content, diverge at that point, and everything after it is cached separately.

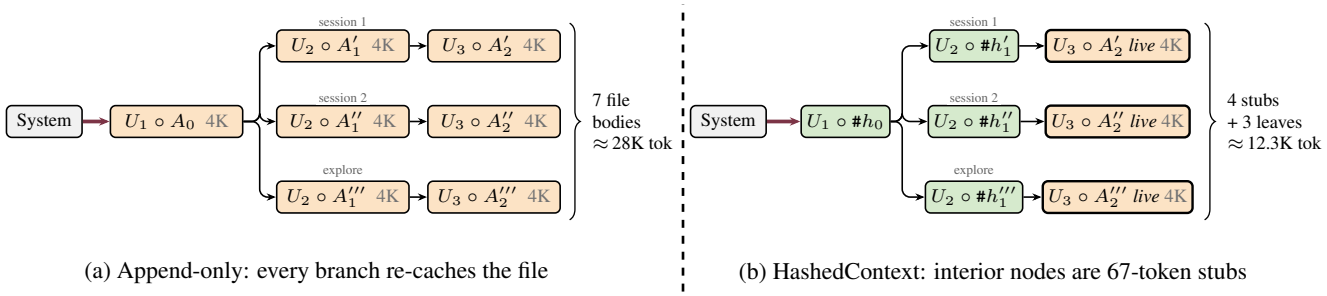
- **Cache Thrashing Under Exploration:** During agentic exploration (e.g., testing alternative fix strategies across branches), every exploration trajectory produces an entirely distinct sequence of file states, each requiring its own KV-cache allocation. Competing exploration paths cannot share cache blocks even when their file states converge.

- **Inter-Session Isolation:** Two agent sessions working on the same repository produce divergent prefix trees because their turn-by-turn file content evolution differs. There is no mechanism for cross-session cache reuse.

The prefix-tree fragmentation manifests as a rapid expansion in the number of distinct cache nodes (or hash entries). Under conventional append-only assembly at turn k, the number of unique prefix nodes grows as:

$$|\text{Nodes}_{\text{conv}}(k)| \geq 1 + \sum_{t=1}^{k}(|\mathcal{T}^{(t)}_{\text{diff}}| + M \cdot \delta_t) \tag{48}$$

where δ_t is an indicator of whether any file was modified at turn t. For a single session, the unique cached content grows linearly with depth, as $k \cdot (C_d + MN)$; the cost is the large per-turn coefficient, and every divergent branch carries its own copies of the file bodies.



(a) Append-only: every branch re-caches the file

(b) HashedContext: interior nodes are 67-token stubs

Figure 5: Radix-tree (prefix-cache) structure for three requests that share a repository: two sessions and one exploration branch, each at turn 3. Bold red edges mark the prefix shared by all three. (a) Under append-only assembly, every divergent node carries a full 4K-token file body, so the tree caches A seven times. (b) Under HashedContext, branches still form wherever hashes differ, but interior nodes hold only the dialogue and a ∼67-token stub; full file bodies appear only once per leaf.

7.3 HashedContext Deterministic Prefix Stabilization

HashedContext reduces the size of divergent branches by replacing variable file content with fixed-length, deterministic stub sequences. Regardless of how file content evolves across turns, the `[HASH_STUB]` block for file f is always:

$$\text{stub}(f) = \underbrace{[\texttt{HASH_STUB}]}_{\text{fixed prefix}}\ \text{path}(f)\ \text{sha256}(\text{content}(f))\ \underbrace{[\texttt{/HASH_STUB}]}_{\text{fixed suffix}} \tag{49}$$

The stub structure itself is constant-length and position-deterministic. The only variable field is the SHA-256 digest, which is itself a fixed-width 64-character hex string. The total stub length is $|\text{stub}| = \sigma \approx 67$ tokens, independent of file content (it varies only with the path length).

Stable Prefix Tree. Under HashedContext, the prefix-tree structure at turn k is:

$$\text{PrefixTree}_{\text{HC}}(k) = \bigcup_{t=1}^{k-1} \underbrace{(\text{TurnPrefix}_t \circ M \times \text{stub}_f)}_{\text{deterministic structure}} \tag{50}$$

Because every stub has identical token count and structural format, the radix tree maintains a nearly constant branching factor across turns. The only new nodes introduced per turn are:

1. The active diff tokens $|\mathcal{T}^{(k)}_{\text{diff}}|$, which are typically under 100 tokens.
2. The authoritative tail $\mathcal{F}_{\text{authoritative}}$, which is identical in structure to the file body in conventional assembly but appears only once.

The live prompt at turn k occupies:

$$\begin{aligned}|\text{Nodes}_{\text{HC}}(k)| &= 1 + (k-1)\cdot(C_d + M\cdot\sigma) \\ &\quad + |\mathcal{T}^{(k)}_{\text{diff}}| + M\cdot N\end{aligned} \tag{51}$$

This grows linearly with k with a small coefficient. One caveat: each previous turn's live tail is a separate leaf that the next turn does not share. Unless those stale leaves are evicted, the tree also accumulates up to $(k-1)\cdot M\cdot N$ tokens of old tails, so the benefit depends on the cache evicting them.

Cross-Session Cache Sharing. When K concurrent agent sessions reference the same repository, their prompts diverge at the first differing token in either architecture. Identical stubs for unchanged files help only if the preceding dialogue is also identical, which is rare for independent sessions: in practice they share the system prompt and little else. In our 16-GPU simulation (512 agents), cross-session reuse was negligible under both architectures (prefix-cache hit rates of 0.3% append-only and 0.6% HashedContext). What HashedContext changes is the size of each session's private branch: interior nodes hold ≈67-token stubs instead of full file bodies. Figure 5 shows both trees for two sessions and one exploration branch over the same repository.

7.4 Multi-Version Exploration Stability

A practical concern in agentic deployments is tree-based exploration: an agent attempts a fix, observes a test failure, branches to an alternative approach, and iterates. Each branch produces a distinct file evolution path. Under conventional assembly, these branches create entirely separate prefix trees, negating any cache sharing between the original trajectory and the exploratory branch—even though they may share the first k turns of identical state.

Under HashedContext, all branches share a common stub prefix up to the divergence point. The radix tree maintains a single shared path for the stub sequence, with branching occurring only at the first differing diff token. Formally, for branches diverging at turn t_0:

$$\text{Prefix}^{(t_0)}_{\text{shared HC}} = \sum_{i=1}^{t_0-1} (C_d + M \cdot \sigma) \quad \text{identical across all branches} \tag{52}$$

This shared prefix represents $(t_0 - 1) \cdot (C_d + M \cdot \sigma)$ tokens of KV-cache that are reused across all exploration branches. Append-only branches also share their common history up to t_0; the difference is that after divergence each HashedContext branch carries stubs rather than full file copies, which reduces the memory each branch adds.

7.5 Quantitative Cache Savings

Consider an agent operating over $M = 5$ files ($N = 4{,}000$) across $P = 30$ turns, with $|\mathcal{T}_{\text{diff}}| = 150$ tokens per turn, $C_d = 500$ and $\sigma = 67$. The conventional resident prompt at turn 30 is:

$$\begin{aligned} \text{PrefixTotal}_{\text{conv}} &\approx \sum_{k=1}^{30} (500 + 5 \cdot 4{,}000 + 150) \\ &= 30 \cdot 20{,}650 = 619{,}500 \text{ tokens} \end{aligned} \tag{53}$$

Under HashedContext:

$$\begin{aligned} \text{PrefixTotal}_{\text{HC}} &= \sum_{k=1}^{30} (500 + 5 \cdot 67 + 150) + 5 \cdot 4{,}000 \\ &= 30 \cdot 985 + 20{,}000 = 49{,}550 \text{ tokens} \end{aligned} \tag{54}$$

The resident prompt at turn 30 is reduced by 92.0%. In physical memory terms, at 160 KB per token:

$$\begin{aligned} \text{Mem}_{\text{conv}} &= 619{,}500 \times 160 \text{ KB} = 99.1 \text{ GB} \\ \longrightarrow \quad \text{Mem}_{\text{HC}} &= 49{,}550 \times 160 \text{ KB} = 7.93 \text{ GB} \end{aligned} \tag{55}$$

This is a 12.5× reduction in the KV-cache needed to hold the turn-30 prompt, which then fits within a single 80 GB GPU. It is not a reduction in everything a prefix cache may store over the session: each earlier turn's 20,000-token live tail is a separate leaf, so if none are evicted, the unique tokens cached over 30 turns total about $30 \cdot 985 + 30 \cdot 20{,}000 = 629{,}550$, no fewer than append-only. The saving relies on stale tails being evicted.

8 Enterprise Operational Safety and Runaway-Loop Cost

Autonomous coding agents, by their nature, operate without continuous human supervision. A critical failure mode for enterprise deployments is the *runaway loop*: an agent trapped in a cyclic retry, test-fix, or debug-verify pattern that generates unbounded token consumption and API costs. In this section, we model the financial blast radius of runaway agent loops. Under both architectures, cost grows quadratically with loop depth; HashedContext reduces the quadratic coefficient by roughly 25–30×, so a given budget allows a far deeper loop and a given depth costs far less.

8.1 The Runaway Loop Failure Model

Consider an autonomous agent executing a software engineering task that enters a failure loop. Common triggers include:

1. **Flaky test cascades:** An agent repeatedly attempts to fix a non-deterministic test failure, producing a new edit hypothesis on every iteration, each of which fails for stochastic reasons.
2. **Linter–syntax error cycles:** An agent generates syntactically incorrect code, the linter rejects it, the agent reformulates, and the cycle repeats. If the agent's fix strategy converges slowly, the loop deepens indefinitely.
3. **Conflicting requirements:** The agent resolves one constraint violation only to create another, oscillating between two unsatisfiable states.

Let P denote the number of turns executed within a failure loop before manual intervention or a budget limit is reached ($P \geq 30$ is common in production incidents). Let M be the number of files in the working set, N the average file token count, C_d the dialogue overhead per turn, and α the cost per token (including both input and output tokens, weighted by the asymmetric pricing of LLM APIs).

8.2 Conventional Cost Acceleration

Under conventional append-only prompt assembly, the cumulative token cost after P turns in a failure loop is:

$$C_{\text{conv}}(P) = \alpha \sum_{k=1}^{P} (k \cdot C_d + k \cdot M \cdot N + G_k) \tag{56}$$

where G_k is the number of generated tokens at turn k. For failure loops, G_k is typically large because the agent produces extensive diagnostic output, test logs, and attempted fixes. Assuming $G_k \approx G$ (constant per turn), the cumulative cost is:

$$C_{\text{conv}}(P) = \alpha \left[\frac{P(P+1)}{2} \cdot (C_d + M \cdot N) + P \cdot G \right] \tag{57}$$

The dominant term scales as $\mathcal{O}(P^2 \cdot M \cdot N)$. For a concrete scenario with $M = 5$ files, $N = 4{,}000$ tokens,

$C_d = 500$, $G = 1{,}000$, and $\alpha = \$2.50$ per million input tokens (illustrative pricing) with $\alpha_{\text{out}} = \$10$ per million output tokens:

$$C_{\text{conv}}(30) = 2.5 \times 10^{-6} \cdot \frac{30 \cdot 31}{2} \cdot (500 + 20{,}000) + 10 \times 10^{-6} \cdot 30 \cdot 1{,}000 \tag{58}$$

$$C_{\text{conv}}(30) = 2.5{\times}10^{-6}{\cdot}465{\cdot}20{,}500{+}10{\times}10^{-6}{\cdot}30{,}000 \tag{59}$$

$$C_{\text{conv}}(30) = \$23.83 + \$0.30 = \$24.13 \tag{60}$$

At $P = 100$ turns (a plausible production escalation scenario):

$$C_{\text{conv}}(100) = 2.5{\times}10^{-6}{\cdot}5{,}050{\cdot}20{,}500{+}10{\times}10^{-6}{\cdot}100{,}000 \tag{61}$$

$$C_{\text{conv}}(100) = \$258.81 + \$1.00 = \$259.81 \tag{62}$$

The cost scales quadratically: increasing the loop depth from 30 to 100 turns (3.3×) multiplies the cost by approximately 10.8×. This quadratic acceleration makes runaway loops financially dangerous for enterprise deployments, where a single unmonitored agent session can accrue hundreds of dollars in minutes.

8.3 HashedContext Runaway-Loop Cost

Under HashedContext, each turn in a failure loop produces a prompt with stub-compressed history and a single authoritative tail. The cumulative cost after P turns is:

$$C_{\text{HC}}(P) = \alpha_{\text{in}} \sum_{k=1}^{P} (k \cdot C_d + (k-1) \cdot M \cdot \sigma + M \cdot N + D_k) + \alpha_{\text{out}} \sum_{k=1}^{P} G_k \tag{63}$$

where D_k is the diff token volume at turn k. In a failure loop where files change minimally (the agent modifies a few lines per iteration), D_k remains small. Let $D_k \approx D$ and $G_k \approx G$:

$$C_{\text{HC}}(P) = \alpha_{\text{in}} \left[\frac{P(P+1)}{2} \cdot C_d + \frac{P(P-1)}{2} \cdot M \cdot \sigma + P \cdot (M \cdot N + D) \right] + \alpha_{\text{out}} \cdot P \cdot G \tag{64}$$

The cost is still quadratic in P: both the dialogue term (C_d) and the stub term ($M \cdot \sigma$) are multiplied by $P^2/2$. The key change is that the quadratic coefficient is $C_d + M\sigma$ (approximately $500 + 5 \times 67 = 835$ tokens) rather than $C_d + MN$ (approximately $500 + 5 \times 4{,}000 = 20{,}500$ tokens), a reduction of about 24.6×. Dialogue does not shrink, so this factor is much smaller than N/σ.

Evaluating the same $P = 30$ scenario with $\sigma = 67$, $D = 100$, $M = 5$:

$$C_{\text{HC}}(30) = 2.5 \times 10^{-6} \big[465 \cdot 500 + 435 \cdot 335 + 30 \cdot (20{,}000 + 100) \big] + 10 \times 10^{-6} \cdot 30{,}000 \tag{65}$$

$$C_{\text{HC}}(30) = 2.5{\times}10^{-6} \left(232{,}500 + 145{,}725 + 603{,}000\right){+}0.30 \tag{66}$$

$$C_{\text{HC}}(30) = 2.5{\times}10^{-6}{\cdot}981{,}225{+}0.30 = \$2.45{+}\$0.30 = \$2.75 \tag{67}$$

At $P = 100$:

$$C_{\text{HC}}(100) = 2.5 \times 10^{-6} \big[5{,}050 \cdot 500 + 4{,}950 \cdot 335 + 100 \cdot 20{,}100 \big] + 1.00 \tag{68}$$

$$C_{\text{HC}}(100) = 2.5{\times}10^{-6} \left(2{,}525{,}000 + 1{,}658{,}250 + 2{,}010{,}000\right){+}1.00 \tag{69}$$

$$C_{\text{HC}}(100) = 2.5{\times}10^{-6}{\cdot}6{,}193{,}250{+}1.00 = \$15.48{+}\$1.00 = \$16.48 \tag{70}$$

The cost ratio between conventional and HashedContext at $P = 100$ is:

$$\frac{C_{\text{conv}}(100)}{C_{\text{HC}}(100)} = \frac{259.81}{16.48} \approx 15.8\times \tag{71}$$

Simulated runaway loops. In our fleet simulation, 4,058 agents were stuck in an edit–test-fail–re-read loop for 100 turns ($M = 2$ files, real repository content). The median cumulative cost at the same pricing was \$4.19 append-only versus \$0.82 HashedContext at $P = 30$, and \$46.71 versus \$6.48 at $P = 100$ (7.2×). The median HashedContext prompt grew by about 424 tokens per turn (append-only: 3,776), and cumulative HashedContext cost rose 3.49× from $P = 50$ to $P = 100$, close to the 4× expected of a quadratic and well above the 2× of a linear trend.

8.4 Asymptotic Blast-Radius Confinement

Figure 6 plots both cost curves together with the budget cut-offs. To establish the formal bound, we compare the asymptotic scaling of both architectures:

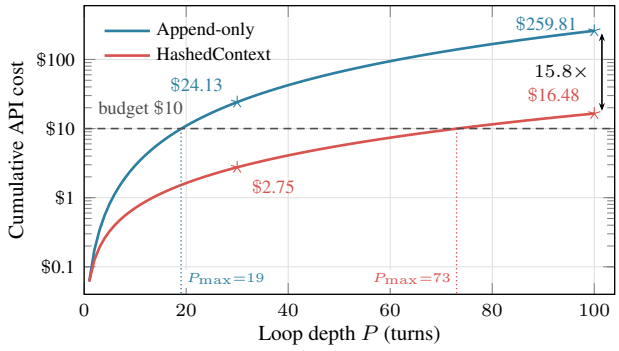


Figure 6: Cumulative cost of a runaway agent loop (Section 8 model: M=5, N=4K, C_d=500, σ=67, \$2.50/M input and \$10/M output tokens; log scale). Both curves are quadratic in P; HashedContext's coefficient is about 24.6× smaller. For a fixed \$10 budget, the append-only agent is cut off at turn 19 and HashedContext at turn 73. At P=100 the gap is 15.8×.

$$C_{\text{conventional}}(P) = \alpha \sum_{k=1}^{P} (k \cdot (C_d + M \cdot N) + G_k) \in \mathcal{O}(P^2 \cdot M \cdot N) \tag{72}$$

$$C_{\text{HC}}(P) = \alpha \sum_{k=1}^{P} \big[kC_d + (k-1)M\sigma + MN + D_k\big] + \alpha_{\text{out}} \sum_{k=1}^{P} G_k \in \mathcal{O}\big(P^2(C_d + M\sigma) + PMN\big) \tag{73}$$

The asymptotic expressions show that both are quadratic and differ in their coefficients:

$$\boxed{C_{\text{conventional}}(P) \in \mathcal{O}\big(P^2(C_d + MN)\big)} \quad \text{vs.} \quad \boxed{C_{\text{HashedContext}}(P) \in \mathcal{O}\big(P^2(C_d + M\sigma)\big)} \tag{74}$$

In the limit $P \to \infty$, the ratio tends to a finite constant:

$$\lim_{P\to\infty} \frac{C_{\text{conv}}(P)}{C_{\text{HC}}(P)} = \frac{C_d + MN}{C_d + M\sigma} \approx 24.6\times \quad (\sigma = 67) \tag{75}$$

or 30.4× with $\sigma = 35$. A compact ≈20-token stub would raise it to about 34×.

Budget-Limited Loop Depth. For a budget B, the deepest loop either architecture can afford is the largest P with $C(P) \le B$. Writing $C_{\text{HC}}(P) = aP^2 + bP$ with $a = \alpha(C_d + M\sigma)/2$ and $b = \alpha(C_d/2 - M\sigma/2 + MN + D) + \alpha_{\text{out}}G$:

$$P_{\max}(B) = \left\lfloor \frac{-b + \sqrt{b^2 + 4aB}}{2a} \right\rfloor \tag{76}$$

For B = \$10 this gives $P_{\max} = 73$ ($\sigma = 67$; 78 with $\sigma = 35$). The same construction with $a = \alpha(C_d + MN)/2$ and $b = \alpha(C_d + MN)/2 + \alpha_{\text{out}}G$ gives $P_{\max} = 19$ for append-only. Both architectures therefore have a closed-form worst-case exposure per session; HashedContext's allows roughly 4× deeper loops for the same budget. In our simulated runaway agents ($M = 2$), the median append-only agent exhausted \$10 after 46 turns, while the median HashedContext agent had not reached it after 100.

8.5 Multi-Agent Fleet Cost Containment

In enterprise deployments with A autonomous agents operating concurrently, the expected aggregate cost over a monitoring window W is:

$$\mathbb{E}[C_{\text{fleet}}] = A \cdot \mathbb{E}[C_{\text{session}}(P)] \cdot \frac{W}{\mathbb{E}[\Delta t]} \tag{77}$$

Under either architecture, session cost is finite for any finite loop depth, but its quadratic growth gives the fleet cost distribution a heavy tail: a few agents in deep loops can dominate the total. HashedContext does not remove the quadratic tail; it shrinks it by the coefficient ratio above. Across our simulated fleet of 20,000 sessions, total cost at the illustrative pricing was \$391,367 append-only, \$162,246 with compaction, and \$45,797 under HashedContext.

9 Datacenter Gateway Ingress and Network Throughput

In production deployments, LLM inference requests traverse a multi-layer network stack before reaching the GPU compute tier. The API gateway—typically implemented via Envoy, NGINX, or LiteLLM—performs TLS termination, HTTP routing, rate limiting, authentication, and request validation. For agentic workloads, the serialization and deserialization of large JSON payloads at the gateway layer constitutes a significant and often overlooked bottleneck. In this section, we model the network and serialization overhead of agent conversation payloads and quantify the relief provided by HashedContext's stub-based compression.

9.1 Gateway Processing Pipeline

A typical inference gateway processes each request through the following pipeline:

1. **TLS Termination:** Decrypt the incoming HTTPS connection, incurring asymmetric crypto overhead proportional to connection count and payload size.

2. **HTTP Header Parsing:** Extract routing metadata, authentication tokens, and content-type headers.

3. **JSON Deserialization:** Parse the request body from JSON into an internal schema representation, validating field types, bounds, and required keys.

4. **Schema Validation:** Validate the parsed payload against an OpenAPI or JSON Schema specification, checking that all required fields (e.g., `messages`, `model`, `max_tokens`) are present and correctly typed.

5. **Rate Limiting and Routing:** Check per-user rate counters, select the target inference engine, and forward the request.

Steps (1)–(4) are CPU-bound and scale with payload size. Under high concurrency, the gateway's CPU cores become the bottleneck, limiting the ingress throughput before GPU capacity is saturated.

9.2 Payload Size Analysis

A conventional agentic request payload at turn k serializes the entire conversation history as a JSON array of message objects. Each message object contains:

- `role` field (system/user/assistant): ≈ 20 bytes
- `content` field: $k \cdot (C_d + M \cdot N)$ tokens, each token averaging ≈ 3.5 UTF-8 bytes in JSON-escaped form
- Structural overhead (braces, quotes, commas, newlines): $\approx 15\%$ of content size

The total payload size at turn k is approximately:

$$\text{Payload}_{\text{conv}}(k) = \gamma \cdot k \cdot (C_d + M \cdot N) \tag{78}$$

where $\gamma \approx 4.0$ bytes per token accounts for JSON overhead and token-to-byte expansion. For $M = 5$, $N = 4{,}000$, $C_d = 500$:

$$\text{Payload}_{\text{conv}}(10) = 4.0 \cdot 10 \cdot (500 + 20{,}000) = 820{,}000 \text{ bytes} \approx 820 \text{ KB} \tag{79}$$

At $k = 30$:

$$\text{Payload}_{\text{conv}}(30) = 4.0 \cdot 30 \cdot 20{,}500 = 2{,}460{,}000 \text{ bytes} \approx 2.46 \text{ MB} \tag{80}$$

These payload sizes are typical of production agentic sessions observed in practice, with individual requests routinely exceeding 300 KB at shallow turns and growing to multiple megabytes for deep interactions.

9.3 HashedContext Payload Compression

Under HashedContext, the stub-compressed history transforms the payload to:

$$\begin{aligned}\text{Payload}_{\text{HC}}(k) = \gamma \cdot \big[&k \cdot C_d + (k-1) \cdot M \cdot \sigma \\ &+ M \cdot N + D_k\big]\end{aligned} \tag{81}$$

Evaluated at $k = 10$ with $\sigma = 67$, $D_k = 150$:

$$\begin{aligned}\text{Payload}_{\text{HC}}(10) &= 4.0 \cdot \big[10 \cdot 500 + 9 \cdot 5 \cdot 67 \\ &\quad + 5 \cdot 4{,}000 + 150\big] \\ &= 4.0 \cdot [5{,}000 + 3{,}015 + 20{,}000 + 150]\end{aligned} \tag{82}$$

$$\text{Payload}_{\text{HC}}(10) = 4.0 \cdot 28{,}165 = 112{,}660 \text{ bytes} \approx 113 \text{ KB} \tag{83}$$

At $k = 30$:

$$\begin{aligned}\text{Payload}_{\text{HC}}(30) &= 4.0 \cdot \big[15{,}000 + 29 \cdot 335 \\ &\quad + 20{,}000 + 150\big] \\ &= 4.0 \cdot [15{,}000 + 9{,}715 + 20{,}000 + 150]\end{aligned} \tag{84}$$

$$\text{Payload}_{\text{HC}}(30) = 4.0 \cdot 44{,}865 = 179{,}460 \text{ bytes} \approx 179 \text{ KB} \tag{85}$$

The compression ratio at $k = 30$ is:

$$\frac{\text{Payload}_{\text{conv}}(30)}{\text{Payload}_{\text{HC}}(30)} = \frac{2{,}460 \text{ KB}}{179 \text{ KB}} \approx 13.7\times \tag{86}$$

Measured on a 30-turn agent session over a real repository, request bodies were 2.35 MB versus 174 KB at $k = 30$ (13.5×). Both payloads grow linearly with k: the conventional one by $C_d + MN$ tokens per turn, the HashedContext one by $C_d + M\sigma$ (measured: 80 KB at $k = 1$ to 174 KB at $k = 30$). The ratio therefore tends to a finite constant:

$$\lim_{k \to \infty} \frac{\text{Payload}_{\text{conv}}(k)}{\text{Payload}_{\text{HC}}(k)} = \frac{C_d + M \cdot N}{C_d + M \cdot \sigma} \approx 24.6\times \tag{87}$$

These savings apply to what the *client* sends, i.e., to edge-native deployment (Section 10). In gateway-native mode, the gateway still receives the full conventional payload.

9.4 CPU Deserialization Throughput

The CPU cost of JSON deserialization at the gateway scales with payload size. Empirical benchmarking on modern server processors (AMD EPYC, Intel Xeon) shows JSON parsing throughput of approximately 200–400 MB/s

per core for typical LLM payloads using simdjson or similar optimized parsers [7].

The time to deserialize a single request at the gateway is:

$$T_{\text{deserialize}}(k) = \frac{\text{Payload}(k)}{R_{\text{parse}}} \tag{88}$$

where R_{parse} is the parsing throughput. For conventional payloads at $k = 30$:

$$T_{\text{deserialize, conv}} = \frac{2.46 \times 10^6}{300 \times 10^6} = 8.2 \text{ ms} \tag{89}$$

For HashedContext payloads at $k = 30$:

$$T_{\text{deserialize, HC}} = \frac{179 \times 10^3}{300 \times 10^6} = 0.60 \text{ ms} \tag{90}$$

The deserialization time is reduced by 13.7× in this model. Measured parse times for the $k = 30$ request bodies above were 7.40 ms versus 0.59 ms with Python's standard `json` module (12.5×) and 2.95 ms versus 0.22 ms with `orjson` (13.4×). Under concurrent load with N_{req} requests per second, the gateway CPU utilization for deserialization is:

$$U_{\text{deserialize}} = N_{\text{req}} \cdot T_{\text{deserialize}} \tag{91}$$

For a gateway with C CPU cores, the maximum sustainable request rate under deserialization-limited conditions is:

$$\text{RPS}_{\text{max, conv}} = \frac{C}{T_{\text{deserialize, conv}}} \approx \frac{32}{0.0082} \approx 3{,}902 \text{ req/s} \tag{92}$$

$$\text{RPS}_{\text{max, HC}} = \frac{C}{T_{\text{deserialize, HC}}} \approx \frac{32}{0.00060} \approx 53{,}300 \text{ req/s} \tag{93}$$

In edge-native mode, HashedContext increases the gateway's deserialization-limited throughput by about 13.7× in this model. Gateway-native mode behaves differently, because the gateway must parse the full conventional payload and then transform it. In a real HTTP load test (64 unmodified clients × 20 turns, 1,280 requests per mode), gateway CPU time per request was 2.9 ms for a pass-through baseline, 9.6 ms for gateway-native HashedContext (2.8 ms parse, 3.5 ms transform, 2.8 ms lossless self-check, 0.5 ms re-serialization), and 0.6 ms for edge-native HashedContext. Request rates were 111, 65 and 107 req/s respectively. Gateway-native mode still cut the prompt tokens reaching the backend by 7.9× (309.9M → 39.4M), which is where its value lies: it moves CPU work onto the gateway in exchange for GPU memory.

9.5 TLS and Socket Buffer Effects

Beyond deserialization, large payloads impact the network stack in three additional ways. As above, these savings apply to edge-native deployment, where the client sends the smaller payload.

TLS Record Processing: TLS 1.3 splits application data into records of up to 16 KB. A 2.46 MB conventional payload requires approximately 150 TLS records, each requiring decryption via AES-GCM (or ChaCha20-Poly1305). A 179 KB HashedContext payload requires 11 records, reducing per-connection decryption overhead by about 14×.

Socket Buffer Memory: The kernel socket buffer for each in-flight request must hold the full payload. For N_{conn} concurrent connections, the total socket memory footprint is:

$$\text{Mem}_{\text{socket}}(k) = N_{\text{conn}} \cdot \text{Payload}(k) \tag{94}$$

Under conventional assembly at $k = 30$ with $N_{\text{conn}} = 1{,}000$:

$$\text{Mem}_{\text{socket, conv}} = 1{,}000 \times 2.46 \text{ MB} = 2.46 \text{ GB} \tag{95}$$

Under HashedContext:

$$\text{Mem}_{\text{socket, HC}} = 1{,}000 \times 179 \text{ KB} = 179 \text{ MB} \tag{96}$$

This 13.7× reduction in socket memory pressure enables higher connection densities per gateway instance without exhausting kernel memory limits.

Network Interface Utilization: At the network interface level, the bandwidth consumed by agent traffic is:

$$\text{BW} = \text{RPS} \cdot \text{Payload}(k) \tag{97}$$

For a 100 Gbps network interface serving 1,000 requests per second with conventional payloads at $k = 30$:

$$\text{BW}_{\text{conv}} = 1{,}000 \times 2.46 \text{ MB/s} = 2.46 \text{ GB/s} \approx 19.7 \text{ Gbps} \tag{98}$$

Under HashedContext:

$$\text{BW}_{\text{HC}} = 1{,}000 \times 179 \text{ KB/s} = 179 \text{ MB/s} \approx 1.4 \text{ Gbps} \tag{99}$$

The network bandwidth consumption drops from 19.7% of a 100 Gbps link to 1.4%, freeing capacity for other traffic and reducing the probability of tail latency spikes due to network congestion under bursty agent load patterns.

10 Deployment Topologies

HashedContext features a bimodal deployment architecture that enables immediate adoption without requiring coordinated ecosystem upgrades:

10.1 Edge-Native Mode (Client-Side)

Implemented directly within agent runtimes and IDE extensions (e.g., VS Code extensions, command-line agents). The client environment computes SHA-256 hashes locally on CPU; hashing a typical turn's files (about 78 KB) took 0.12 ms in our measurements.

- **Advantage:** Reduces network transmission payloads and directly lowers API token costs for the user before request dispatch.

10.2 Gateway-Native Mode (Server-Side Ingress)

Deployed as an intelligent reverse-proxy filter (e.g., Envoy filter or LiteLLM plugin) positioned in front of inference serving engines (vLLM, SGLang, TensorRT-LLM).

- **Advantage:** Solves the cold-start ecosystem adoption problem. Unmodified legacy clients can send large 150K-token prompts; the gateway deduplicates historical turns on CPU prior to tensor scheduling.
- **Impact:** The inference provider unilaterally reclaims GPU high-bandwidth memory across all incoming streams without requiring third-party client upgrades. In our load test, backend prompt tokens fell 7.9×.
- **Cost:** The gateway still receives and parses the full payload, and the transformation adds CPU work: 9.6 ms per request versus 2.9 ms for pass-through in our load test (Section 9). Network and deserialization savings at ingress require edge-native mode.

Selecting Mode A or Mode B. Either topology can run either prompt mode. A gateway is well placed to choose per session, because it knows whether a session's requests are routed to the same prefix cache and how heavily loaded the backend is: Mode B for sticky, lightly loaded routes, and Mode A otherwise (Table 7).

11 Evaluation

We evaluate HashedContext in three parts: a simulated validation at scale (Section 11.1), a dual-mode comparison of Mode A and Mode B with scenario recommendations (Section 11.2), which together produce every number in the abstract, and a single pilot session on a production endpoint (Section 11.3).

11.1 Simulated Validation

Setup. A reference implementation of HashedContext (hash gate, vault, stubs, Algorithm 1, $\mathcal{R}(h)$, and an OpenAI-format gateway filter) was driven by 20,000 scripted coding agents working on four real open-source repositories (`flask`, `express`, `zod`, `axios`) at pinned commits. Agents read real files, apply real edits, and run real checks. Three behaviour profiles were used: *paper* (re-reads all $M = 5$ files every turn), *realistic* (lazy, partial re-reads) and *runaway* (stuck in an edit–test-fail loop for 100 turns). Every prompt was built under append-only, Mode A and Mode B (with and without rebasing) and counted exactly with the `cl100k_base` tokenizer, 824,728 turns in total. GPU serving was modelled with a simulator that implements a vLLM-style paged KV pool, automatic prefix caching over real token IDs with LRU eviction, continuous batching, and roofline timing with the H100 constants of Section 5 (80 GB HBM, 3.35 TB/s, 30 GB of FP8 weights, 163,840 bytes of KV per token). The comparison baseline in the simulator is append-only with post-hoc compaction at 128K tokens, since plain append-only does not fit.

Losslessness. All 11,032 sampled sessions reconstructed byte-exact from stubs and vault, all 2,061,819 live tails matched the current file and its digest, and all 1,280 gateway requests in the HTTP load test round-tripped exactly. For Mode B, all 547,785 diffs reproduced the exact new file bytes, and all 5,516 sampled prompts rebuilt every file byte-exact from the prompt text alone.

Tokens and KV memory. Table 1 gives the mean KV-cache needed per request (mean prompt length × 163,840 bytes). Plain append-only does not fit in the 50 GB KV pool for the five-file profile, so compaction is the fair baseline. The measured stub size was $\sigma = 67.8$ tokens on average.

Profile	Append-only	+ compaction	HashedContext	vs append	vs compact.
All	30.96 GB	11.28 GB	3.50 GB	88.7%	69.0%
Paper	40.73 GB	13.39 GB	4.78 GB	88.3%	64.3%
Realistic	14.14 GB	8.94 GB	1.81 GB	87.2%	79.7%
Runaway	39.26 GB	12.18 GB	4.23 GB	89.2%	65.3%

Table 1: Mean KV-cache needed per request across 824,728 simulated turns.

Serving. Table 4 reports a simulated 16-GPU cluster serving 512 agents with prefix caching on. The KV pool was 99.7% full in all modes, so memory was the binding limit. Under Mode A, about 75% less KV per running request let about 4× as many requests run at once; because each decode step reads the weights once for the whole batch, this raised agent turns per hour 3.57×. The baseline's 204 compaction calls and longer prompts also slowed it, and the simulator cannot fully separate those effects from the memory effect.

Compute. Whether HashedContext saves compute depends on the cache regime and the mode (Table 2). With an ideal per-session prefix cache that never evicts, Mode A computes 3.6% *more* prompt tokens than append-only, because each turn's live tail must be recomputed once it becomes a stub, while Mode B computes 88.1% fewer. Under load (Table 4), the cache is evicted and Mode A computes 59% fewer prompt tokens per turn than the compaction baseline, Mode B 55% fewer; with no prefix cache, Mode A computes 88.7% fewer than append-only and Mode B 84.8% fewer.

Change detection. In a mechanical needle test (Table 6; 300 cases; one line of one file changed externally before the final turn; the grader may use only the prompt text), Mode A identified the changed file in 100% of cases but the changed line in 0%; adding $\mathcal{R}(h)$ raised both to 100% at a cost of 15% more prompt tokens. Mode B identified file, line and new text in 100% of cases without $\mathcal{R}(h)$, at 25,158 prompt tokens on average versus 24,723 for Mode A. This test measures what information each prompt contains, not how well a live model uses it.

Diff-chain test (live LLM). To probe Mode B's main risk, we built a test in which a stuck agent edits the same file every turn, so the file reaches 10 edits, and asked a live model (`deepseek-v4.1-flash` via an OpenAI-compatible endpoint, temperature 0, 30 cases) about the file's current content. Each prompt layout was described accurately in the system prompt. The question names a unique line of the current file and asks for the line immediately after it. Under Mode A and append-only the model can find the line by text; under Mode B it must apply the whole diff chain, whose hunks locate changes only by line number. Table 8 reports the results. Mode B answered 27 of 30 correctly with a 10-diff chain (90.0%) and 28 of 30 with `max_diffs_before_rebase` = 5, which leaves a 4-diff chain at the end (93.3%), against 28 of 30 for Mode A (93.3%) and 29 of 30 for append-only (96.7%). One case failed under every layout. The Mode B-specific failures were cases 6 and 11 at $N = 10$ and case 15 at $N = 5$, so the gap between modes is one or two cases and the 95% intervals overlap almost entirely. We therefore find no clear "lost in the middle" penalty at 10 diffs for this model, but the sample can only detect large effects. For accuracy-critical work we recommend $N = 5$, which costs about 16% more prompt tokens in this test (10,546 versus 9,064).

A first version of the test asked for the text of line N instead. That version mostly measured line counting: Mode A, with the complete current file in view, answered only 4 of 30 correctly, append-only 9, and Mode B 17, likely because its hunk headers print line numbers. We report it for completeness but do not use it to compare modes. A mechanical reader answered every case correctly under every layout, which confirms the harness. The test is included as `llm_eval.chain_eval`.

Limitations. The agents are scripted policies, not LLMs, so token, byte and cost accounting is exact but reasoning quality is not tested. The live diff-chain test covers one model and 30 cases, so it can detect only large differences between modes. GPU results come from a simulator using H100 constants, not physical hardware.

11.2 Dual-Mode Comparison and Scenario Recommendations

Tables 2–7 compare the two modes. Table 2 gives memory and compute across the simulated fleet, and Table 3 breaks them down by workload profile. Table 4 reports the 16-GPU cluster, and Table 5 reports a single GPU across serving scenarios: with and without prefix caching, and from 1 to 64 agents per GPU.

The pattern is consistent. Mode A uses the least KV memory in every profile, and it wins whenever the prefix cache cannot stay warm: without caching (Mode B completes 0.66–0.91× the turns of Mode A) and at 16 or more agents per GPU, where memory pressure evicts the cache (Mode B 0.80–0.91×; cluster 0.90×). Mode B wins when the cache stays warm: at 1–8 agents per GPU with prefix caching it completes 1.07–1.76× the turns of Mode A, and it computes 88.1% fewer prompt tokens than append-only in the warm-cache accounting, which matters for APIs that discount cached input. Its rebase bound capped every diff chain at 10 diffs, compared with up to 99 without rebasing, at a cost of 0.73 GB more KV per request. Table 8 reports the live diff-chain test, and Table 7 turns these measurements into recommendations.

11.3 Pilot Session on a Production Endpoint

We also ran a single multi-stage refactoring session on DeepSeek V4 using Mode A via its OpenAI-compatible REST endpoint, harvesting `prompt_tokens`, `prompt_cache_hit_tokens`, and generation latency from the API usage payloads. The task edited a TypeScript module (`revert.ts`, 417 lines, $\approx 4,000$ tokens) over five interaction cycles. Being a single session, it illustrates the mechanism but does not establish an effect size.

Table 9 presents the telemetry from the pilot session. Figure 7 shows the analytical model of Section 4 for the same file size.

Three points qualify this table. First, the conventional run's cache-hit tokens equal 97.8% of its turn-4 prompt, consistent with Section 4.3: the saving is in tokens held and billed, not in cache reuse. Second, the HashedContext run produced 3.3× more output tokens; at output prices four times input prices, the extra 984 output tokens cost about as much as 3,900 input tokens, so end-to-end cost savings in this session were smaller than the prompt reduction suggests. Third, each architecture was run once, so the 57.67% figure should be read as an illustration; the simulated fleet (Table 1) measured a median of 72.8% at turn 4 for five-file sessions.

11.4 Reasoning Fidelity and Needle-in-a-Haystack Verification

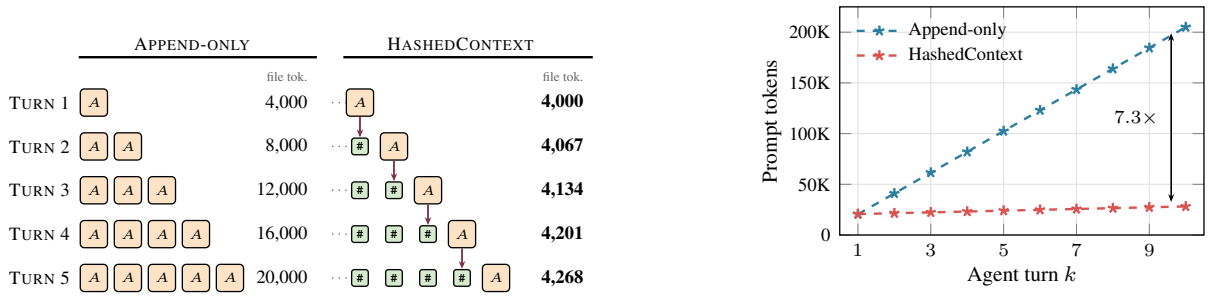
A critical concern with context compression is degradation in model reasoning fidelity: does stubbing historical files impair model comprehension or prevent fine-grained diff tracking?

During the evaluation, an intentional micro-mutation was applied to the source file at line 416, altering a single trailing comment from:

```
// This is an edited comment by Architect
```

Mode	KV / request	vs append	vs compaction	Compute, warm cache	Compute, no cache	Cache hit
Append-only	30.96 GB	—	+174.5%	—	—	95.9%
Append-only + compaction	11.28 GB	-63.6%	—	+144.8%	-63.6%	72.6%
Mode A: Authoritative Tail	3.50 GB	-88.7%	-69.0%	+3.6%	-88.7%	62.6%
Mode B: Lazy Tail, rebase after 10 diffs	4.71 GB	-84.8%	-58.2%	-88.1%	-84.8%	96.8%
Mode B without rebasing	3.98 GB	-87.1%	-64.7%	-89.4%	-87.1%	96.6%

Table 2: Memory and compute across 20,000 simulated sessions (824,728 turns). Compute = prompt tokens the server must process. Warm cache = ideal per-session prefix cache that never evicts.



(a) File residency per turn

(b) Prompt length per turn (model)

Figure 7: (a) Residency of one 4K-token file over five turns (analytical, σ=67). Each live copy (A) is demoted to a stub (#) on the next turn, so five turns hold 20,000 file tokens append-only versus 4,268 under HashedContext (−78.7%). (b) Analytical prompt length with M=5 files, N=4K, σ=67, C_d=500: $S_{\text{conv}}(k) = k(C_d{+}MN)$ versus $S_{\text{HC}}(k) = kC_d + (k{-}1)M\sigma + MN$.

	KV		Compute, warm		Compute, no cache	
Profile	A	B	A	B	A	B
Paper	-88.3%	-88.1%	+2.4%	-91.5%	-88.3%	-88.1%
Realistic	-87.2%	-80.3%	+5.1%	-84.0%	-87.2%	-80.3%
Runaway	-89.2%	-84.7%	+4.6%	-85.1%	-89.2%	-84.7%
All	-88.7%	-84.8%	+3.6%	-88.1%	-88.7%	-84.8%

Table 3: Change vs append-only by workload profile. KV = mean KV-cache per request; with no cache, compute equals prompt length, so it matches the KV column.

Metric	Append-only + compaction	Mode A	Mode B
Agent turns per hour	3,883	13,854	12,456
p95 turn latency (s)	689	190	210
p95 time to first token (s)	655	107	128
Requests running at once	3.54	14.06	12.84
KV per running request (GB)	14.10	3.55	3.88
KV pool utilisation	99.7%	99.7%	99.7%
Prefix-cache hit rate	0.3%	0.6%	0.6%
Prompt tokens computed per turn	43,069	17,462	19,414
Generated tokens/s (cluster)	335	824	739
Compaction calls	204	0	0

Table 4: Simulated 16-GPU H100 cluster serving 512 agents with prefix caching on (1 hour, paper's H100 constants).

Prefix cache	Agents/GPU	Baseline	Mode A	Mode B	B vs A
on	1	297	434	587	1.35×
on	2	514	829	1,118	1.35×
on	4	295	1,168	2,054	1.76×
on	8	182	1,303	1,398	1.07×
on	16	197	949	756	0.80×
on	32	266	943	829	0.88×
on	64	331	1,023	930	0.91×
off	1	86	304	202	0.66×
off	2	135	457	324	0.71×
off	4	180	666	440	0.66×
off	8	178	773	590	0.76×
off	16	195	836	675	0.81×
off	32	267	942	827	0.88×
off	64	334	1,022	926	0.91×

Table 5: Agent turns completed per GPU-hour on one simulated H100 by serving scenario.

Prompt	Tokens	File	Line	Text
Append-only	217,963	100%	100%	100%
Mode A (stubs + live tail)	24,723	100%	0%	0%
Mode A + $\mathcal{R}(h)$	28,443	100%	100%	100%
Mode B (rebase after 10)	25,158	100%	100%	100%
Mode B (no rebase)	25,158	100%	100%	100%

Table 6: Mechanical needle test: what each prompt contains (300 cases). The grader is a program, so it does not measure whether a live model can apply diff chains.

Scenario	Mode	Evidence
Prefix cache on, light load (1–8 agents per GPU, cache stays warm)	Mode B	B completes 1.07–1.76× the turns of A (Table 5)
Prefix cache on, heavy load (16+ agents per GPU, cache evicted)	Mode A	B completes 0.80–0.91× the turns of A; cluster: 0.90× (Table 4)
No prefix cache, or requests not routed to the same cache	Mode A	B completes 0.66–0.91× the turns of A (Table 5)
Billed API with discounted cached input tokens	Mode B	B computes 88.1% fewer prompt tokens than append-only with a warm cache; A computes +3.6% (Table 2)
Lowest KV memory per request	Mode A	A 3.50 GB vs B 4.71 GB per request (Table 2)
Long edit loops, accuracy-critical work	Mode A, or Mode B with N=5	Live test, line after anchor (Table 8): A 93.3%, B 90.0% with 10 diffs, 93.3% with N=5; differences of 1–2 of 30 cases, within noise
Model must see exactly what changed, without retrieval	Mode B	B shows the changed lines directly; A needs $\mathcal{R}(h)$ for that (Table 6)

Table 7: Recommended deployment mode by serving scenario, derived from the simulated measurements and the live diff-chain test.

Question	Prompt	Diffs	Correct	95% CI
Line after anchor	Append-only	0	29/30 (96.7%)	83%–99%
Line after anchor	Mode A	0	28/30 (93.3%)	79%–98%
Line after anchor	Mode B, N=10	10	27/30 (90.0%)	74%–97%
Line after anchor	Mode B, N=5	4	28/30 (93.3%)	79%–98%
Line number	Append-only	0	9/30 (30.0%)	17%–48%
Line number	Mode A	0	4/30 (13.3%)	5%–30%
Line number	Mode B, N=10	10	17/30 (56.7%)	39%–73%

Table 8: Live-LLM diff-chain test on deepseek-v4.1-flash (30 cases; the target file has 10 edits). *Line after anchor*: report the line that follows a given unique line. *Line number*: report line N, which mostly measures line counting. 95% Wilson intervals.

Metric	Conventional	HashedContext	Change
Prompt tokens (turn 4)	27,747	11,746	−57.67%
Prompt-cache hit tokens	27,136	10,752	—
Hit tokens / turn-4 prompt	97.8%	91.5%	—
Output tokens	436	1,420	+226%

Table 9: Pilot session telemetry on DeepSeek V4 (single session).

to:

```
// This is an removed comment by Architect
```

Although four preceding turns referenced historical `[HASH_STUB]` blocks, the model identified the single-line comment mutation, distinguished logic from cosmetic comments, and produced a 10-point audit spanning 1,420 output tokens. This is a single qualitative observation. The live tail contained the complete current file, so the model could read the new line directly. Detecting a change from stubs alone is a harder test: as the mechanical needle test in Section 11.1 shows, stubs reveal which file changed but not what changed unless $\mathcal{R}(h)$ supplies the previous version. Section 11.1 reports a first live-model test of Mode B's diff chains; a systematic live-LLM evaluation of reasoning quality is future work.

12 Related Work

Efficient LLM Serving and Memory Management. PagedAttention [1] introduced virtual memory paging to mitigate external fragmentation in GPU KV-cache allocation. FlashAttention [2] optimized GPU SRAM tiling to minimize memory bandwidth bottlenecks during attention computation. SGLang [6] introduced RadixAttention, a prefix-tree-based caching mechanism that shares KV-cache blocks across requests with common prefixes. HashedContext occupies a complementary layer in the serving stack, forming a hierarchical architecture:

$$\text{Agent Runtime} \longrightarrow \textbf{HashedContext} \longrightarrow \textbf{RadixAttention} \longrightarrow \text{GPU HBM3} \tag{100}$$

In this stack, PagedAttention and RadixAttention manage *physical memory allocation*: they partition GPU VRAM into non-contiguous blocks, eliminate external

fragmentation, and enable on-demand memory sharing across sequences through tree-structured or hash-based block tables. HashedContext operates upstream as a *temporal admission controller*: it eliminates sequence-level redundancy *before* memory allocation occurs, ensuring that only unique, authoritative content is presented to the memory manager. The two systems are complementary rather than competing: RadixAttention cannot reduce the input sequence length, and HashedContext cannot manage physical block allocation. Together, they form a full-stack solution where HashedContext reduces the input to its irreducible semantic core, and RadixAttention maps that core efficiently into physical VRAM. FlashAttention further optimizes the *execution kernel* by tiling attention computations across GPU SRAM, operating downstream of both layers.

Context Window Optimization. Prior approaches have explored prompt compression through soft prompts, attention sinks, and recursive summarization. However, these techniques are inherently lossy. HashedContext is lossless regarding the authoritative file state under standard cryptographic assumptions (SHA-256 collision resistance), guaranteeing exact source-code representations at the generation frontier.

Autonomous Agent Safety and Cost Bounding. The problem of runaway agent costs has been addressed in production systems through budget-enforcing middleware, turn limits, and token caps. These mechanisms impose hard cutoffs that truncate agent trajectories mid-task, causing incomplete work and wasted prior investment. HashedContext offers a complementary approach: rather than capping execution, it reduces the quadratic cost coefficient by roughly 25–30×, so deep trajectories stay affordable for longer before any cap is reached.

13 Conclusion

We have presented **HashedContext**, a content-addressed prompt architecture that sharply slows the context growth of multi-turn autonomous coding agents, and framed it as a **Dual-Mode Architecture**. Mode A (Authoritative Tail) replaces historical file copies with SHA-256 stubs and keeps one complete current copy at the prompt tail. Mode B (Lazy Tail) keeps history append-only and sends changes as line diffs, resending a file in full after at most 10 diffs. Both modes are lossless: every simulated session was recoverable byte-exact.

In simulation, Mode A reduces the KV-cache an agent needs by 88.7% compared with append-only context (69.0% compared with append-only plus compaction), and Mode B by 84.8% (58.2%). With a warm prefix cache, Mode B computes 88.1% fewer prompt tokens than append-only while Mode A computes 3.6% more. On a simulated 16-GPU H100 cluster serving 512 agents, Mode A completes 3.57× and Mode B 3.21× more agent turns per hour than append-only with compaction. Neither mode dominates: Mode B leads when the prefix cache stays warm (up to 1.76× Mode A's turns at light load), and Mode A leads under heavy load, without a cache, and when the model should always see one complete current copy. Table 7 gives scenario-based recommendations, and a gateway can apply them per session.

Our analysis and simulated validation show that HashedContext:

- Reduces the KV-cache read per decoding step in proportion to prompt length. Once model-weight reads are included, a single request's step time improves 1.39× (100K → 20K tokens); the larger gain is aggregate throughput from larger batches, about 5× at full memory under the H100 constants (Section 5);
- In Mode A, leaves a single authoritative copy of each file in the prompt, removing stale versions the model could otherwise attend to; in Mode B, shows exactly which lines changed without retrieval. Whether either improves reasoning accuracy remains to be measured with a live LLM (Section 6);
- Shrinks the resident prompt at the final turn by about 12.5× in the example of Section 7. Append-only prompts already reach 89–91% prefix-cache hit rates in simulation, so the benefit is lower KV residency rather than higher cache reuse; cross-session reuse was not observed;
- Reduces the quadratic coefficient of runaway-loop cost by $(C_d + MN)/(C_d + M\sigma) \approx 25$–$30\times$. Cost remains $\mathcal{O}(P^2)$ but grows far more slowly: simulated stuck agents cost 7.2× less at $P = 100$ (Section 8);
- In edge-native mode, reduces request payloads about 13× at turn 30 (measured), relieving gateway deserialization and network load. In gateway-native mode the gateway still parses the full payload and its CPU time per request rises about 3×, while backend prompt tokens fall 7.9× (Section 9).

HashedContext requires zero modifications to underlying transformer weights or inference execution engines, operating as a non-invasive architecture for both client agents and server-side gateways. The serving results above come from a simulator using H100 constants rather than physical hardware, and the agents are scripted rather than LLM-driven. The most important open question is Mode B's diff chains. In a first live test on one model, Mode B scored 90.0% with 10-diff chains and 93.3% with $N = 5$, against 93.3% for Mode A, differences within the noise of 30 cases. Until larger, multi-model tests are run, we recommend $N = 5$ for accuracy-critical work. Validating reasoning fidelity with live models for both modes and measuring on real GPUs are the next steps.

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[5] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS)*, pages 483–485, 1967.

[6] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, and Hao Zhang. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.

[7] Geoff Langdale and Daniel Lemire. Parsing gigabytes of json per second. *The VLDB Journal*, 28(6):941–960, 2019.

[8] Zhaofeng Wu, Xinyan Zhao, Yilun Hua, Yushi Hu, and Heng Ji. Recursively summarizing long sequences for efficient llm inference. *arXiv preprint arXiv:2311.12325*, 2023.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12:523–540, 2023.